

ESTUDIANTE:

Cabrera, Santiago
smcabrera@fi.uba.ar

110445

Índice

1. Resumen	2
2. Parte I: transistor de canal largo	2
2.1. Parámetros de la estructura MOS	2
2.2. Curva $\psi_{s,d}$ vs. V_{DS}	3
2.3. Cálculo numérico de la curva de salida I_D vs. V_{DS}	3
2.4. Cálculo numérico de la curva de transferencia I_D vs. V_{GS}	4
2.5. Resultados y análisis	4
3. Parte II: transistor de canal corto	9
3.1. Escalamiento y parámetros de la estructura MOS	9
3.2. Modulación del largo del canal	9
3.3. Reducción de la barrera inducida por <i>drain</i> (D.I.B.L)	10
3.4. Degradación de la movilidad	10
3.5. Saturación de la velocidad de arrastre	11
3.6. Análisis de calidad del transistor obtenido	11
3.7. Resultados y análisis	12
4. Conclusiones	16
A. Apéndice: Código Parte I	17
B. Apéndice: Código Parte II	26

1. Resumen

El presente trabajo consiste de dos partes. En la primera se obtendrá de forma numérica la relación entre la tensión de superficie en el terminal del *drain* $\psi_{s,d}$ y la tensión de salida V_{DS} para el caso de un transistor MOSFET canal N. En base a este resultado se simularán las curvas de salida I_D vs. V_{DS} y de transferencia I_D vs. V_{GS} . También se simularán dichas curvas siguiendo los modelos cuadrático y clásico, los cuales serán comparados con el resultado numérico para determinar qué tan acertados son.

En la segunda parte se propone reducir el tamaño del transistor siguiendo la guía de escalado a campo constante y estudiar cómo los efectos de canal corto modifican la curva de salida, así como determinar cuál de estos efectos predomina por sobre los otros. Para esto se hará uso de la aproximación cuadrática, sobre la cual se simularán los siguientes efectos de canal corto: modulación del largo del canal, D.I.B.L., degradación de la movilidad y saturación de velocidad de arrastre.

2. Parte I: transistor de canal largo

En esta sección se estudian 3 aproximaciones distintas para computar las curvas de salida y transferencia de un transistor MOSFET canal N con gate de polisilicio y óxido de silicio a una temperatura de 300 K (lo que implica un $V_{th} = 25,875$ mV).

n_i [cm ⁻³]	N_a [cm ⁻³]	E_g [eV]	$\epsilon_{r_{si}}$
1×10^{10}	6×10^{15}	1,12	11,7

Tabla 2.1: Características del sustrato.

$\epsilon_{r_{ox}}$	t_{ox} [nm]	$E_0 - E_{f_{poly}}$ [eV]	L [µm]	W [µm]	r_j [nm]	V_{DD} [V]
3,9	35	4,04	1,5	16	420	6,5

Tabla 2.2: Parámetros constructivos de la estructura MOS.

Las tablas 2.1 y 2.2 resumen los valores utilizados en los cálculos realizados en las secciones siguientes. En las secciones 2.1 a 2.4 se desarrolla el contenido teórico y en la Subsección 2.5 se presentan los resultados.

2.1. Parámetros de la estructura MOS

En esta sección se detallan las ecuaciones utilizadas para calcular los parámetros de la estructura MOS necesarios para el desarrollo del trabajo. Los parámetros a calcular son: μ_{n_p} ; C_{ox} ; ψ_B ; ϕ_{poly} ; ϕ_s ; $\phi_{bi}(GB)$; V_{FB} ; γ ; V_T y m .

Para calcular el efecto que tiene la concentración de impurezas en la movilidad se utilizará la siguiente ecuación:

$$\mu(N_I) = \mu_{min} + \frac{\mu_{max} - \mu_{min}}{1 + (N_I/N_{ref})^\alpha}. \quad (1)$$

Los valores típicos para silicio tipo P a $T = 300$ K son $\mu_{min} = 68,5$ cm²/Vs, $\mu_{max} = 1414$ cm²/Vs, $N_{ref} = 9,2 \times 10^{16}$ cm⁻³ y $\alpha = 0,711$.

$$C_{ox} = WL \frac{\epsilon_{r_{ox}} \epsilon_0}{t_{ox}} \quad (2)$$

$$\psi_B = -V_{th} \ln \left(\frac{N_a}{n_i} \right) \quad (3)$$

$$\phi_{poly} = \frac{E_0 - E_{f_{poly}}}{q} \quad (4)$$

$$\phi_s = \chi_{si} + \frac{E_g}{2q} - \psi_B \quad (5)$$

$$\phi_{bi} = \phi_s - \phi_{poly} \quad (6)$$

$$V_{FB} = -\phi_{bi} \quad (7)$$

$$\gamma = \frac{\sqrt{2qN_a\epsilon_{r_{si}}\epsilon_0}}{C'_{ox}} \quad (8)$$

$$V_T = V_{FB} - 2\psi_B + \gamma\sqrt{-2\psi_B} \quad (9)$$

$$m = 1 + \frac{\gamma}{2\sqrt{-2\psi_B}} \quad (10)$$

Las ecuaciones 2 a 10 permiten calcular el resto de parámetros necesarios. Los resultados de las mismas se presentarán en la Subsección 2.5.

2.2. Curva $\psi_{s,d}$ vs. V_{DS}

Uno de los objetivos de este trabajo es estudiar cómo varía el doblamiento de las bandas de energía en el terminal del *drain* cuando varía la tensión aplicada V_{DS} . Dicho de otra forma, se busca analizar cómo impacta V_{DS} en la tensión de superficie en el *drain*, $\psi_{s,d}$. Es decir, se busca la función $\psi_{s,d}(V_{DS})$. Para hallar la misma, se plantea un conjunto de ecuaciones que resolverán el problema de forma numérica.

$$\mathcal{E}_{s,d}^2 = \frac{2qV_{th}N_a}{\epsilon_s} \left[\exp\left(-\frac{\psi_{s,d}}{V_{th}}\right) + \frac{\psi_{s,d}}{V_{th}} - 1 + \frac{n_i^2}{N_a^2} \left(\exp\left(-\frac{V_{DS}}{V_{th}}\right) \left(\exp\left(\frac{\psi_{s,d}}{V_{th}}\right) - 1 \right) - \frac{\psi_{s,d}}{V_{th}} \right) \right] \quad (11)$$

$$Q'_s(\psi_{s,d}) \approx 2qV_{th}N_a\epsilon_s \left[\frac{\psi_{s,d}}{V_{th}} + \frac{n_i^2}{N_a^2} \exp\left(\frac{\psi_{s,d} - V_{DS}}{V_{th}}\right) \right] \quad (12)$$

La Ecuación 11 muestra la dependencia del campo eléctrico en la interfaz óxido - semiconductor en el terminal del *drain* con V_{DS} y $\psi_{s,d}$. Si se asume que $\psi_{s,d} \gg V_{th}$ y que $\frac{n_i^2}{N_a^2} \ll 1$, la misma puede simplificarse a la ecuación Ecuación 12 aplicando la ley de Gauss $\mathcal{E} = -\frac{Q'}{\epsilon}$.

Paralelamente, se tiene la siguiente ecuación:

$$Q'_s = -C'_{ox}(V_{GS} - V_{FB} - \psi_{s,d}) \quad (13)$$

a partir de la cual se definirá la función $f(\psi_{s,d})$ como

$$\begin{aligned} f(\psi_{s,d}) &= Q_s'^2(\psi_{s,d}) - C_{ox}'^2(V_{GS} - V_{FB} - \psi_{s,d})^2 = \\ &= 2qV_{th}N_a\epsilon_s \left[\frac{\psi_{s,d}}{V_{th}} + \frac{n_i^2}{N_a^2} \exp\left(\frac{\psi_{s,d} - V_{DS}}{V_{th}}\right) \right] - C_{ox}'^2(V_{GS} - V_{FB} - \psi_{s,d})^2. \end{aligned} \quad (14)$$

Se observa que la función f depende de $\psi_{s,d}$ y que V_{GS} y V_{DS} son parámetros de la misma. Además, de la Ecuación 13 se tiene que $f(\psi_{s,d})$ debe ser 0 siempre. De esta manera, para cada par de valores (V_{DS}, V_{GS}) habrá un único valor $\psi_{s,d}$ que satisfaga esta condición, con lo cual resolviendo $f(\psi_{s,d}) = 0$ para cada dupla se podrá construir la relación $\psi_{s,d}(V_{DS})$ para cada valor de V_{GS} deseado. Obsérvese además que la función f tiene derivada conocida:

$$\frac{\partial f}{\partial \psi_{s,d}}(\psi_{s,d}) = 2qN_a\epsilon_s \left[1 + \frac{n_i^2}{N_a^2} \exp\left(\frac{\psi_{s,d} - V_{DS}}{V_{th}}\right) \right] + 2C_{ox}'^2(V_{GS} - V_{FB} - \psi_{s,d}). \quad (15)$$

De esta forma, conociendo la función (14) y su derivada (15) puede hallarse su raíz a través del método Newton-Raphson para cada par de valores (V_{DS}, V_{GS}).

2.3. Cálculo numérico de la curva de salida I_D vs. V_{DS}

Se busca computar la relación I_D vs. V_{DS} para cada valor de V_{GS} usando 3 aproximaciones distintas: la aproximación numérica, la aproximación cuadrática y la aproximación clásica.

El método numérico consiste en hallar I_D a partir de la siguiente ecuación:

$$I_D = \mu_n \frac{W}{L} \left\{ C'_{ox}(V_{GS} - V_{FB} + V_{th})\psi_s - \frac{1}{2}C'_{ox}\psi_s^2 - \frac{2}{3}\sqrt{2\epsilon_s q N_a} \psi_s^{3/2} + V_{th}\sqrt{2\epsilon_s q N_a} \psi_s \right\} \bigg|_{\psi_s=\psi_{s,s}}^{\psi_s=\psi_{s,d}}. \quad (16)$$

Esta ecuación depende de V_{GS} y de V_{DS} (a través de $\psi_{s,d}$ y $\psi_{s,s}$) y es válida para todas las regiones de operación del MOSFET. Para evaluarla, primero es necesario calcular $\psi_{s,d}$ y $\psi_{s,s}$ a partir del método desarrollado en la Subsección 2.2.

Por otro lado, la aproximación cuadrática divide la curva de salida en 3 regiones como se describe a continuación:

$$I_D(V_{DS}, V_{GS}) = \begin{cases} \mu_n C'_{ox} \frac{W}{L} (m-1) V_{th}^2 \exp\left(\frac{V_{GS}-V_T}{mV_{th}}\right) \left[1 - \exp\left(\frac{-V_{DS}}{V_{th}}\right) \right] & \text{si } V_{GS} < V_T \text{ (subumbral)} \\ \mu_n C'_{ox} \frac{W}{L} (V_{GS} - V_T - \frac{m}{2} V_{DS}) V_{DS} & \text{si } V_{DS} < V_{DS(sat)} = \frac{V_{GS}-V_T}{m} \text{ (lineal)} \\ \mu_n C'_{ox} \frac{W}{L} \frac{1}{2m} (V_{GS} - V_T)^2 & \text{si } V_{DS} \geq V_{DS(sat)} \text{ (saturación).} \end{cases} \quad (17)$$

Por último, también se calculó la corriente según la aproximación clásica, la cual utiliza la misma fórmula que la aproximación cuadrática, solo que considera $m = 1$ y no tiene en cuenta la región subumbral (por lo que $I_D = 0$ para $V_{GS} < V_T$). Se graficaron las curvas de salida según las 3 aproximaciones y se compararon entre sí. Los resultados se encuentran en la Subsección 2.5.

2.4. Cálculo numérico de la curva de transferencia I_D vs. V_{GS}

Similar a lo hecho en la sección anterior, en esta se comparan las curvas de transferencia I_D vs. V_{GS} predichas por las 3 aproximaciones. En este caso se hará especial énfasis en la región subumbral ya que en ella los modelos difieren.

2.5. Resultados y análisis

Parámetro	Valor	Unidad
μ_n	1245,1	$\text{cm}^2/\text{V}\cdot\text{s}$
C_{ox}	23,68	fF
ψ_B	-344,26	mV
ϕ_{poly}	4,04	V
ϕ_s	4,91	V
ϕ_{bi}	0,87	V
V_{FB}	-0,87	V
γ	0,452	$\text{V}^{1/2}$
V_T	0,194	V
m	1,272	—

Tabla 2.3: Resumen de los parámetros calculados.

La tabla 2.3 resume los resultados obtenidos de las ecuaciones 1 a 10. En primer lugar se observa que la movilidad se reduce aproximadamente un 12 % respecto del valor natural del silicio debido al efecto del dopaje. Por otro lado, se observa que $V_{FB} < 0$ V, por lo que incluso para el caso $V_{GS} = 0$ V el capacitor MOS se encontrará en vaciamiento, lo que contribuye a reducir el valor de V_T . Se observa también que la tensión umbral es menor que los valores de V_{GS} usados en la simulación, por lo que el transistor siempre estará en la región lineal o de saturación. Por último, se observa que el valor de m difiere de 1 en casi un 30 %, lo que contribuirá a generar discrepancias entre el modelo cuadrático y clásico.

Curva $\psi_{s,d}$ vs. V_{DS}

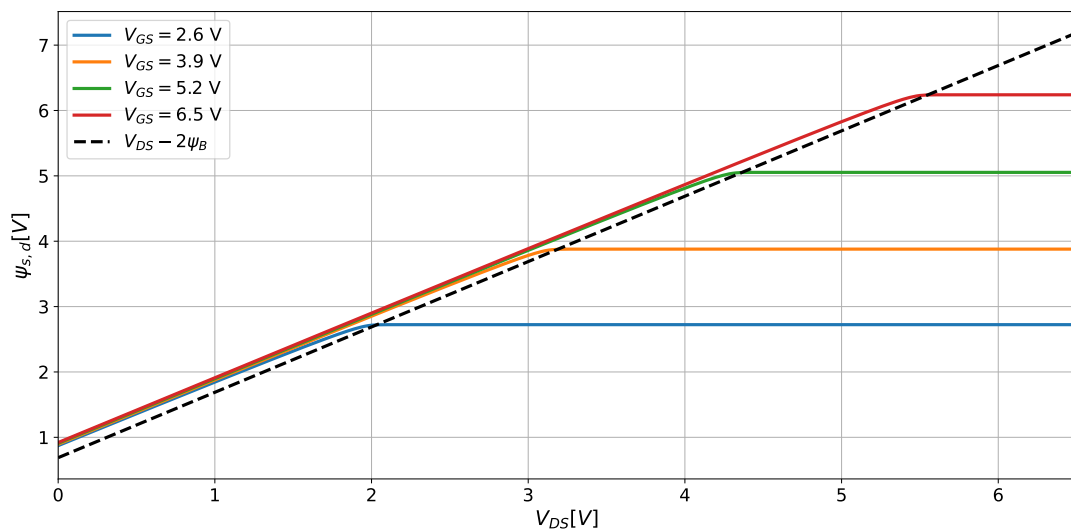


Figura 1: $\psi_{s,d}$ vs. V_{DS} para distintos valores de V_{GS} .

La Figura 1 muestra el resultado de aplicar el método Newton-Raphson a la función $f(\psi_{s,d})$ para cada par de valores (V_{DS}, V_{GS}) , como se explicó en la sección 2.2, obteniendo así una curva $\psi_{s,d}$ vs. V_{DS} para cada valor de V_{GS} . Además, se graficó en línea punteada la función $\psi_{s,d} = V_{DS} - 2\psi_B$, la cual aproxima a todas las curvas en un intervalo de V_{DS} hasta llegar al punto de saturación.

Este gráfico, junto con la Ecuación 16 explican buena parte del funcionamiento del MOSFET: en la región lineal, a medida que V_{DS} aumenta también aumenta $\psi_{s,d}$, por lo que aumenta I_D según la ecuación 16. Sin embargo, cuando $\psi_{s,d}$ satura, también lo hace I_D , y el transistor se encuentra entonces en saturación. También es de interés notar que el valor de $V_{DS(sat)}$ aumenta con mayor V_{GS} . Esto se debe a que, al ser V_{GS} mayor en el canal hay una carga de inversión mayor, por lo que, para contrarrestarla y alcanzar el punto de *pinch-off* se necesita un valor mayor de V_{DS} .

Curvas de salida

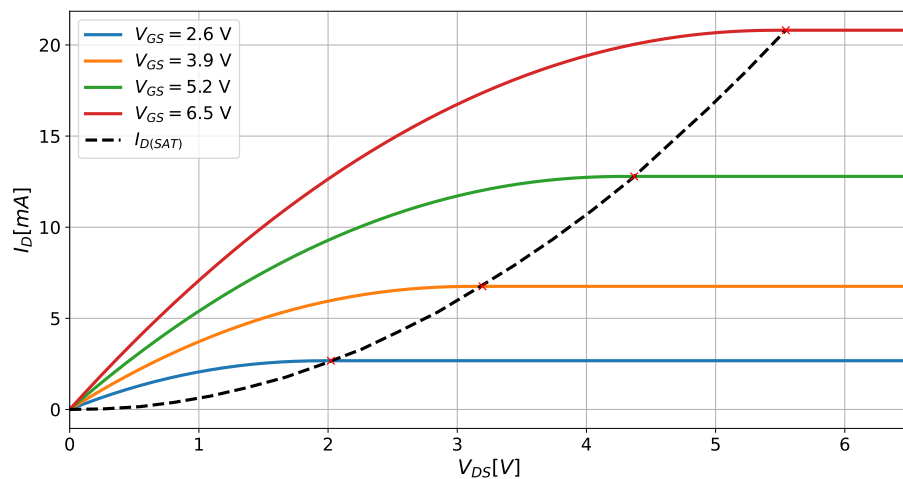


Figura 2: Aproximación numérica de la curva de salida.

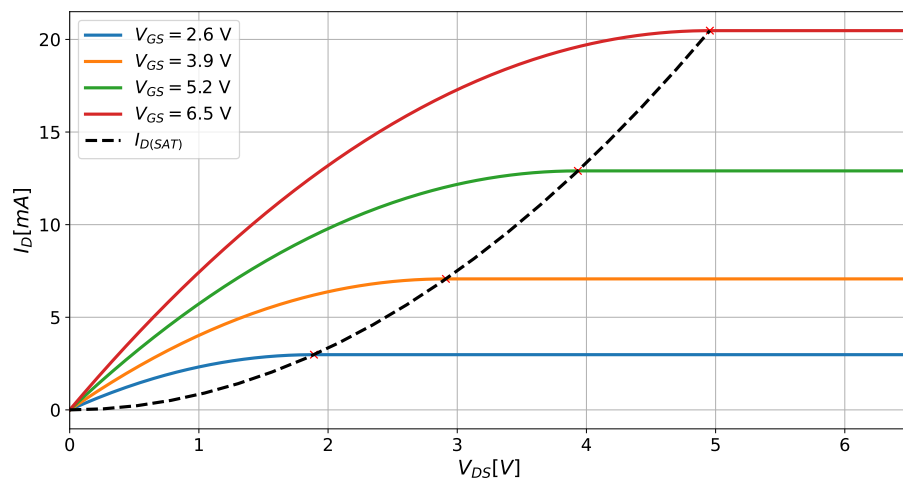


Figura 3: Aproximación cuadrática de la curva de salida.

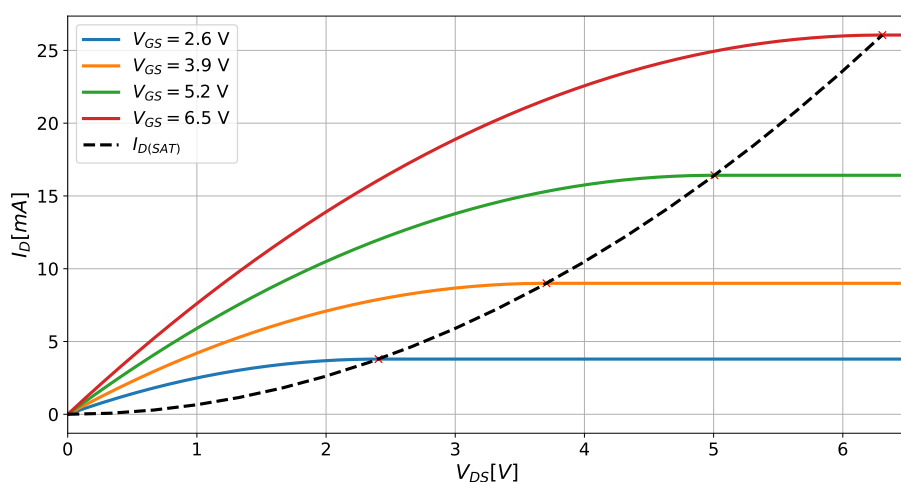
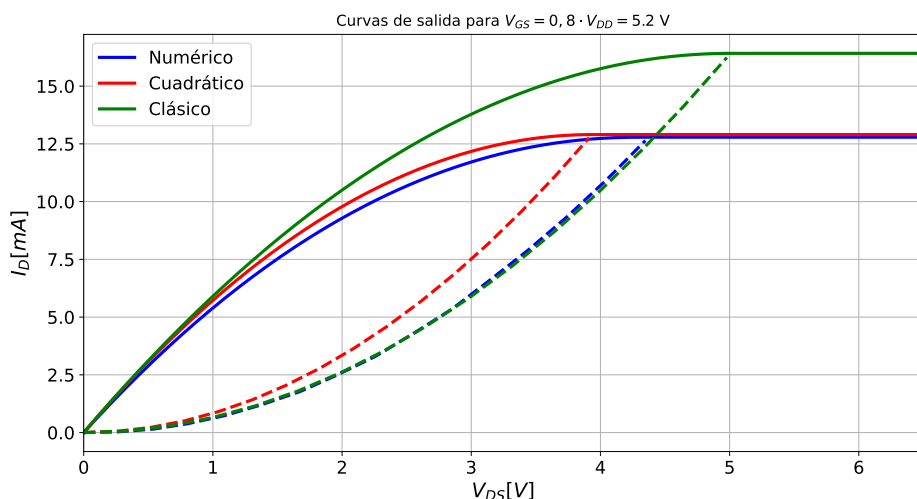


Figura 4: Aproximación clásica de la curva de salida.

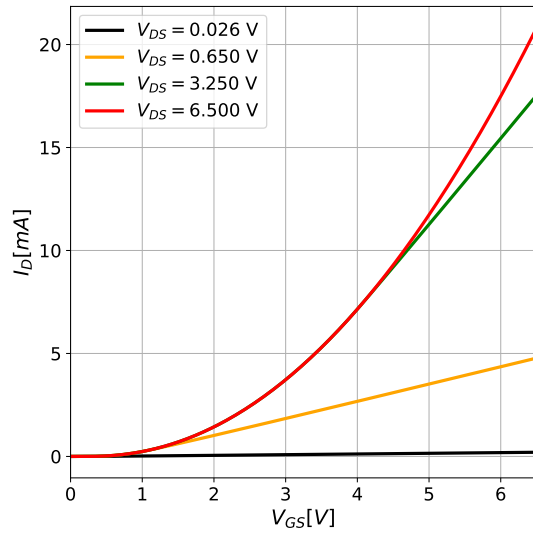
Figura 5: Comparación de las 3 aproximaciones para $V_{GS} = 5,2 \text{ V}$.

Las figuras 2, 3 y 4 muestran las curvas de salida según las aproximaciones numérica, cuadrática y clásica, respectivamente. Los valores de V_{GS} usados fueron $V_{GS} = \{2,6 \text{ V}; 3,9 \text{ V}; 5,2 \text{ V}; 6,5 \text{ V}\}$, mientras que para V_{DS} se generó un vector de 300 puntos con valores entre 0 y V_{DD} . En cada gráfico se incluye en línea negra punteada la curva de saturación $I_{D(sat)}$ vs. $V_{DS(sat)}$ según cada modelo. La curva de saturación del modelo numérico se obtiene hallando la intersección de las curvas $\psi_{s,d}(V_{DS})$ con la recta $\psi_{s,d} = V_{DS} - 2\psi_B$, mientras que para el modelo cuadrático primero se calcula $V_{DS(sat)} = \frac{V_{GS} - V_T}{m}$ para luego calcular $I_{D(sat)}$ con ese valor. El modelo clásico es como el cuadrático, solo que asume $m = 1$.

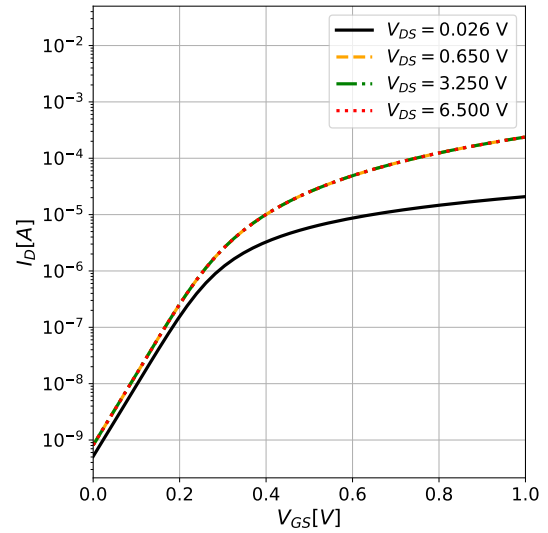
En primer lugar se observa que a medida que V_{DS} crece la corriente I_D crece de forma cuadrática hasta llegar a su valor de saturación, a partir del cual se mantiene constante por más que V_{DS} aumente. El valor de V_{DS} en el cual se produce la saturación así como la corriente de saturación dependen de V_{GS} , de manera que cuanto mayor sea esta tensión mayor serán $V_{DS(sat)}$ e $I_{D(sat)}$. Este es el comportamiento esperado de un MOSFET, el cual se ve reflejado en los tres modelos.

Por otro lado, la Figura 5 compara las 3 aproximaciones con sus respectivas curvas de saturación. Se observa que las aproximaciones numérica y clásica son similares, y que su mayor diferencia está en sus curvas de saturación, ya que $V_{DS(sat)}$ es menor en el modelo cuadrático que en el numérico. Por otro lado, el modelo clásico y el modelo numérico difieren ampliamente ya que $m \approx 1$ es una mala aproximación para el valor de m obtenido en la tabla 2.3. De esta forma, el modelo clásico sobrestima la corriente, como se puede deducir de la Ecuación 17.

Curvas de transferencia

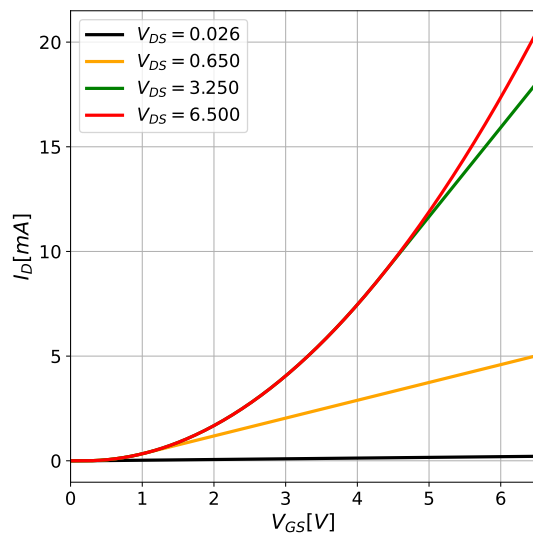


(a) Escala lineal.

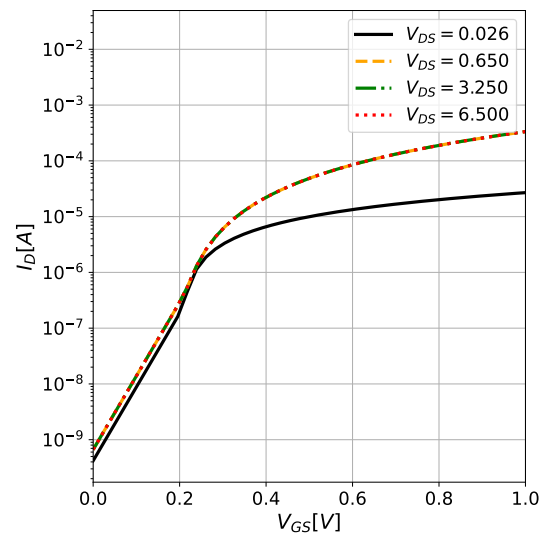


(b) Escala semilogarítmica (región subumbral).

Figura 6: Aproximación numérica de la curva de transferencia.

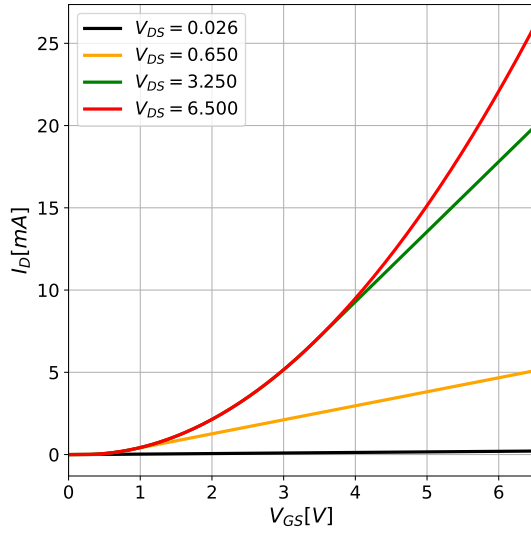


(a) Escala lineal.

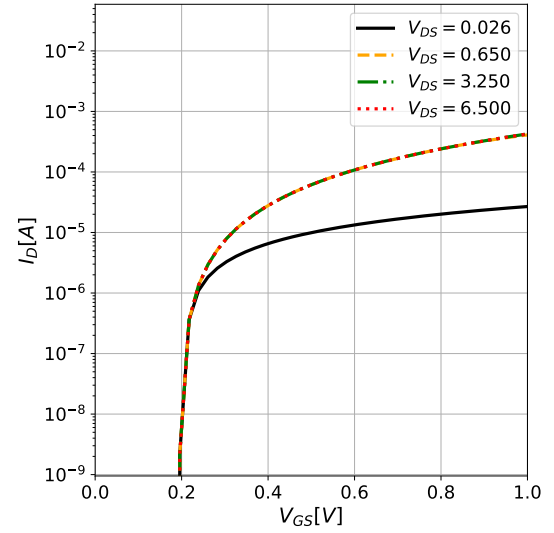


(b) Escala semilogarítmica (región subumbral).

Figura 7: Aproximación cuadrática de la curva de transferencia.

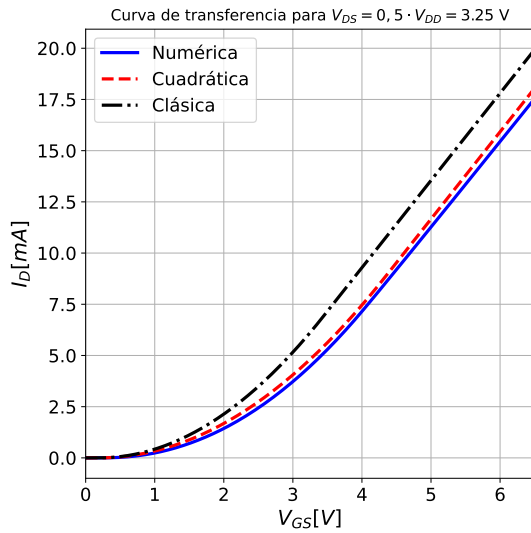


(a) Escala lineal.

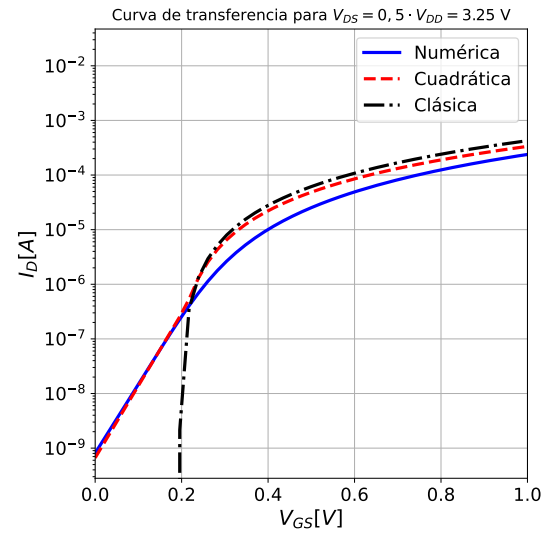


(b) Escala semilogarítmica (región subumbral).

Figura 8: Aproximación clásica de la curva de transferencia.



(a) Escala lineal.



(b) Escala semilogarítmica (región subumbral).

Figura 9: Comparación de las 3 aproximaciones para $V_{DS} = 3,25$ V.

Las figuras 6, 7 y 8 muestran las curvas de transferencia indicadas por los modelos numérico, cuadrático y clásico, respectivamente. Para el modelo numérico se utilizó la Ecuación 16, para lo cual primero fue necesario hallar $\psi_{s,d}$ por el método ya explicado, mientras que para el modelo cuadrático se utilizó la función de la Ecuación 17. Cabe aclarar que el modelo cuadrático puede producir discontinuidades en el punto $V_{GS} = V_T$, por lo que, para evitarlas, se calculó I_{sub} para el intervalo $0 < V_{GS} < V_T$ según 17 y para $V_{GS} > V_T$ se la mantuvo constante. Esta corriente I_{sub} fue luego añadida a la corriente I_D calculada según 17, evitando así discontinuidades en el punto $V_{GS} = V_T$. Para el modelo clásico se utilizó la misma ecuación, solo que con $m = 1$ y con $I_D = 0$ para la región subumbral. Además, se graficaron las mismas curvas en escala semilogarítmica y limitadas al intervalo $V_{DS} = \{0\text{ V}; 1\text{ V}\}$ para poder observar la región subumbral con mayor claridad. En este caso, los valores de V_{DS} utilizados fueron $V_{DS} = \{25,875\text{ mV}; 0,65\text{ V}; 3,25\text{ V}; 6,5\text{ V}\}$, mientras que para V_{GS} se utilizó un vector de 300 puntos con valores entre 0 y V_{DD} .

En todos los casos se observa que para $V_{DS} = V_{th} = 25,875\text{ mV}$ la corriente es despreciable comparada con los otros 3 casos de V_{DS} . También se observa que, a medida que V_{GS} aumenta el transistor pasa de corte a saturación y luego a triodo. Cuanto menor es V_{DS} más angosta es la región de saturación

en la curva de salida, pero también se produce una corriente menor. Este es el comportamiento general esperado de un MOSFET, en el cual los 3 modelos coinciden.

Por otro lado, la Figura 9 compara los 3 modelos para el caso $V_{DS} = 3,25$ V, en escala lineal y semi-logarítmica. En la Figura 9a puede verse el mismo resultado que en la sección anterior: la aproximación clásica sobrestima la corriente debido a que $m \approx 1$ difiere demasiado del parámetro real del transistor, mientras que la aproximación cuadrática y la numérica presentan resultados similares. Ahora además puede verse en la Figura 9b que los resultados numérico y cuadrático coinciden también en la región subumbral y que en la misma existe de hecho una corriente no nula no contemplada por el modelo clásico.

3. Parte II: transistor de canal corto

Parámetros del Dispositivo	Factor Multiplicativo ($\kappa > 1$)
Dimensiones del dispositivo (t_{ox}, L, W)	$1/\kappa$
Concentración de impurezas (N_a, N_d)	κ
Tensión (V)	$1/\kappa$

Tabla 3.1: Parámetros de escalado en MOSFET.

En la segunda parte de este trabajo se propone reducir el tamaño del transistor estudiado en la Parte I en un factor κ . Se seguirán las indicaciones de la Tabla 3.1 para escalar las dimensiones, concentraciones y tensiones del dispositivo y se volverán a calcular los parámetros dados por las ecuaciones 1 a 10. Se estudiará cómo esto impacta en la curva de salida I_D vs. V_{DS} y se simularán también los siguientes efectos de canal corto: Modulación del largo del canal, efecto D.I.B.L., degradación de la movilidad y saturación de la velocidad de arrastre. Se estudiará cómo cada efecto impacta en la curva de salida y se los combinará para analizar cuál es el que tiene mayor impacto en la misma. Por último, se analizará la calidad del transistor obtenido.

3.1. Escalamiento y parámetros de la estructura MOS

Siguiendo las indicaciones de la Tabla 3.1 se escalan las dimensiones, dopajes y tensiones del dispositivo considerando $\kappa = 10$.

t_{ox} [nm]	L [μ m]	W [μ m]	r_j [nm]	N_a [cm^{-3}]	V_{DD} [V]
3,5	0,15	1,6	42	6×10^{16}	0,65

Tabla 3.2: Parámetros escalados.

En base a estos valores se recalcularán los parámetros de la Tabla 2.3 para realizar las simulaciones.

3.2. Modulación del largo del canal

El efecto de modulación del largo del canal (E.M.L.C.) se produce debido a la juntura que se forma entre el *drain* y el *bulk* (juntura DB). En el modo de operación normal del MOSFET la juntura DB se encuentra en inversa, por lo que se forma una zona de vaciamiento con un ancho que se denotará por x_{db} . Si el largo del canal L fuera de un orden de magnitud similar al de x_{db} , la zona de vaciamiento podría reducir L de forma apreciable, lo que tendería a aumentar la corriente.

Para estudiar este efecto de forma cuantitativa es necesario calcular la tensión de contacto de la juntura DB, $\phi_{bi,db}$. La concentración de impurezas donoras en los terminales del *source* y del *drain* es $N_d = 1 \times 10^{20} \text{cm}^{-3} > N_c$, por lo que se trata de un semiconductor degenerado. De esta forma, $\phi_{bi,db}$ se calcula como:

$$\phi_{bi,db} = \frac{E_g}{2q} + V_{th} \ln \left(\frac{N_a}{n_i} \right) = 936,8 \text{ mV}. \quad (18)$$

Por otro lado, el ancho de la zona de vaciamiento x_{db} puede calcularse como

$$x_{db} = \sqrt{\frac{2\epsilon_{si}}{qN_a}(\phi_{bi,db} + V_{DS})} \quad (19)$$

y si se supone que el efecto solo ocurre para $V_{DS} > V_{DS(sat)}$, la variación en el largo del canal ΔL puede calcularse como

$$\Delta L = \sqrt{\frac{2\epsilon_s}{qN_a}} \left(\sqrt{\phi_{bi,db} + V_{DS}} - \sqrt{\phi_{bi,db} + V_{DS(sat)}} \right). \quad (20)$$

Obsérvese que la Ecuación 20 depende de V_{DS} y de $V_{DS(sat)}$, y que esta última depende de V_{GS} , por lo que será necesario calcular un ΔL para cada par de valores (V_{DS}, V_{GS}) deseado. Por último, ya que la corriente es inversamente proporcional al largo del canal, se aplicará un factor de corrección a la corriente producida sin considerar este efecto, lo cual se hará según la siguiente fórmula:

$$I_D = \frac{L}{L - \Delta L} \cdot I_{D(cl)} \quad (21)$$

donde $I_{D(cl)}$ representa la corriente de *drain* del transistor de canal largo.

3.3. Reducción de la barrera inducida por *drain* (D.I.B.L)

El efecto D.I.B.L. tiene un origen similar al E.M.L.C. ya que se produce cuando las zonas de vaciamiento de las junturas SB y DB entran en contacto, lo que genera un camino para que los electrones circulen de *source* a *drain*. Otra forma de entender este efecto es considerar que a medida que aumenta V_{DS} , la energía en el *drain* disminuye, lo que hace disminuir todas las energías a lo largo del canal. Si el canal fuera lo suficientemente corto, esto haría disminuir la barrera de potencial de la juntura SB y permitiría la circulación de electrones.

Para estudiar este efecto de forma cuantitativa se supondrá que las junturas SB y DB son iguales, de forma tal que $\phi_{bi, sb} = \phi_{bi, db}$. También se supondrá que el ancho de la zona de vaciamiento de la juntura SB es igual al ancho de la zona de vaciamiento de la juntura DB en el equilibrio, es decir: $x_{sb} = x_{db0}$. De esta forma, se puede usar la Ecuación 19 con $V_{DS} = 0$ V para calcular x_{db0} , de lo cual se obtiene $x_{db0} = 0,144 \mu\text{m}$.

También es necesario calcular la tensión de *punch-through*, la cual se define como la tensión V_{DS} para la cual las zonas de vaciamiento ocupan todo el largo del canal, es decir $L = x_{db} + x_{sb} = x_{db} + x_{db0}$. Si a esta ecuación se aplica 19 y se despeja V_{DS} se obtiene

$$V_{PT} = \frac{qN_a}{2\epsilon_{si}} (L - x_{db0})^2 - \phi_{bi,db} = -0,962 \text{ V}. \quad (22)$$

Dado que la tensión V_{PT} es negativa, se tiene que para cualquier valor de $V_{DS} > 0$ V habrá una corriente de D.I.B.L. apreciable. Esto es consistente con el hecho de que la zona de vaciamiento de la juntura SB ocupa casi todo el canal ($x_{sb} = 0,144 \mu\text{m}$, $L = 0,15 \mu\text{m}$), de forma tal que ambas zonas de vaciamiento juntas ya son más largas que el canal. Por lo tanto, se tendrá una corriente de D.I.B.L. I_{DIBL} paralela a la corriente de *drain* I_D , incluso para los casos en los que $V_{GS} = 0$ V. Esta corriente de D.I.B.L. se calculará como:

$$I_{DIBL} = WL \frac{9\epsilon_{si}\mu_n}{8L^3} V_{DS}^2. \quad (23)$$

3.4. Degradación de la movilidad

Inicialmente se asumió que la movilidad dependía solamente de la temperatura y de la concentración de impurezas, por lo que podía ser calculada utilizando la Ecuación 1, la cual es válida para silicio a $T = 300$ K. Sin embargo, a medida que L se reduce empiezan a aparecer otros efectos, como variaciones apreciables en el campo eléctrico perpendicular al canal $\mathcal{E}_x$, cargas presentes en la interfaz óxido-semiconductor o irregularidades en la superficie de dicha interfaz.

Para considerar estos efectos de manera cuantitativa, en este trabajo se hará uso de resultados experimentales, según los cuales la movilidad de los portadores depende principalmente del campo eléctrico dentro del canal. De esta forma, se tiene que la movilidad efectiva viene dada por:

$$\mu_{eff} = \mu_0 \left(\frac{\mathcal{E}_{eff}}{\mathcal{E}_0} \right)^\alpha \quad (24)$$

donde se define el campo efectivo $\mathcal{E}_{eff}$ como

$$\mathcal{E}_{eff} = \frac{\int_0^{x_i} \mathcal{E}_x(x) n(x) dx}{\int_0^{x_i} n(x) dx} \quad (25)$$

donde $n(x)$ es la concentración de portadores. A los efectos de este trabajo se considera $\mathcal{E}_0 = 20 \text{ kV/cm}$, $\mu_0 = 975 \text{ cm}^2/(\text{Vs})$ y $\alpha = -0,28$.

La Ecuación 25 puede desarrollarse para hallar una expresión en la que $\mathcal{E}_{eff}$ depende del punto de polarización del transistor.

$$\mathcal{E}_{eff} = \frac{\epsilon_{ox}}{t_{ox}\epsilon_{si}}(V_T - V_{FB} + 2\psi_B) + \frac{\epsilon_{ox}}{2t_{ox}\epsilon_{si}}(V_{GS} - V_T) \quad (26)$$

De esta forma, se tendrá un $\mathcal{E}_{eff}$ para cada valor de V_{GS} utilizado, así como una movilidad distinta para cada uno de dichos valores.

3.5. Saturación de la velocidad de arrastre

El último efecto a estudiar en este trabajo es la saturación de la velocidad de arrastre. En un semiconductor la velocidad de arrastre crece linealmente con el campo eléctrico, pero esto se cumple solo hasta cierto punto, a partir del cual dicha velocidad satura. El campo a partir del cual esto ocurre se llama campo crítico, y en este trabajo se considera que tiene un valor de $\mathcal{E}_c = 27 \text{ kV/cm}$.

Para estudiar este efecto de forma cuantitativa se hará uso de resultados experimentales, según los cuales la velocidad de arrastre depende del campo eléctrico en el canal $\mathcal{E}_y$ según la ecuación

$$v_a(\mathcal{E}_y) \approx \frac{\mu_{eff}\mathcal{E}_y}{1 + \frac{\mathcal{E}_y}{\mathcal{E}_c}} = \frac{\mu_{eff}\mathcal{E}_c\mathcal{E}_y}{\mathcal{E}_c + \mathcal{E}_y} = \mu_{ch}\mathcal{E}_y. \quad (27)$$

A partir de esta ecuación se puede deducir una fórmula para la corriente I_D que contemple esta dependencia de la velocidad con el campo. La misma es:

$$I_D = \frac{C'_{ox}\mu_{eff}\frac{W}{L}}{1 + \frac{V_{DS}}{L\mathcal{E}_c}} \left(V_{GS} - V_T - \frac{m}{2}V_{DS} \right) V_{DS}. \quad (28)$$

De esta fórmula, se obtiene una nueva expresión para $V_{DS(sat)}$.

$$V_{DS(sat)} = L\mathcal{E}_c \left[\sqrt{1 + \frac{2(V_{GS} - V_T)}{mL\mathcal{E}_c}} - 1 \right] \quad (29)$$

En base a la Ecuación 29 se puede analizar qué efecto predomina en la corriente de saturación. Se asume en primer lugar que $\frac{V_{GS}-V_T}{m} \ll L\mathcal{E}_c$. Luego, se puede aplicar la aproximación $\sqrt{1+x} - 1 \approx x/2$ (válida para pequeños valores de x) a la Ecuación 29 para obtener $V_{DS(sat)} = \frac{V_{GS}-V_T}{m}$, que es la tensión de saturación del modelo cuadrático causada por el *pinch-off*. Introducir este resultado en la Ecuación 28 produce $I_{D(sat)} = \frac{C'_{ox}\mu_{eff}}{2m} \frac{W}{L} (V_{GS} - V_T)^2$, que es la corriente de saturación del modelo cuadrático.

Si en cambio se asume que $\frac{V_{GS}-V_T}{m} \gg L\mathcal{E}_c$ puede aplicarse la aproximación $\sqrt{1+x} - 1 \approx \sqrt{x}$ (válida para grandes valores de x) a la Ecuación 29 para obtener $V_{DS(sat)} = \sqrt{2L\mathcal{E}_c \frac{V_{GS}-V_T}{m}}$. Si se aplica este resultado a la Ecuación 28 se obtiene que $I_{D(sat)} = C'_{ox}W\mu_{eff}\mathcal{E}_c(V_{GS} - V_T)$, donde el producto $\mu_{eff}\mathcal{E}_c$ es la velocidad de saturación. Es decir, en este último caso, la corriente satura antes por la saturación de la velocidad de arrastre que por *pinch-off*.

3.6. Análisis de calidad del transistor obtenido

Como último punto de este trabajo se propone analizar qué tan bueno resulta el transistor obtenido. Si bien este análisis dependerá de la aplicación a la cual esté destinado el dispositivo, se supondrá que este se utilizará en un dispositivo V.L.S.I. digital, en el cual el transistor conmutará entre los estados encendido y apagado. Se tiene entonces un comportamiento similar al de una llave, en el cual son deseables tres características: 1) una conmutación clara y rápida entre los estados, 2) una baja resistencia de salida y 3) una baja corriente de fuga en el estado cerrado.

3.7. Resultados y análisis

Escalamiento de parámetros

Parámetro	Valor original	Valor escalado	Unidad
μ_n	1245,1	842,7	$\text{cm}^2/\text{V}\cdot\text{s}$
C_{ox}	23,68	2,37	fF
ψ_B	-344,26	-403,8	mV
ϕ_{poly}	4,04	4,040	V
ϕ_s	4,91	4,974	V
ϕ_{bi}	0,87	0,943	V
V_{FB}	-0,87	-0,934	V
γ	0,452	0,143	$\text{V}^{1/2}$
V_T	194	2,4	mV
m	1,272	1,080	—

Tabla 3.3: Parámetros calculados luego del escalamiento.

En la Tabla 3.3 se vuelcan los resultados de las ecuaciones 1 a 10 luego de escalar las dimensiones, dopajes y tensiones del transistor. En primer lugar se observa que la movilidad disminuye alrededor de un 30 %, lo que se atribuye al aumento en la concentración de impurezas. La capacidad C_{ox} disminuye $\kappa = 10$ veces debido a que todas las dimensiones del transistor (t_{ox} , W y L) disminuyen 10 veces. Por otro lado, las tensiones ψ_B , ϕ_s , ϕ_{bi} y V_{FB} varían aproximadamente 60 mV, lo que se explica analizando la dependencia logarítmica de ψ_B con N_a (Ecuación 3). También se observa que γ y m disminuyen su valor, lo que se justifica analizando la dependencia de estos con el resto de parámetros según las ecuaciones 8 y 10. Por último, se tiene una reducción notable en la tensión V_T , la cual se reduce casi 100 veces. Este es uno de los efectos de canal corto y causará problemas a la hora de controlar la carga de inversión en el canal con la tensión V_{GS} .

Curva de salida I_D vs. V_{DS}

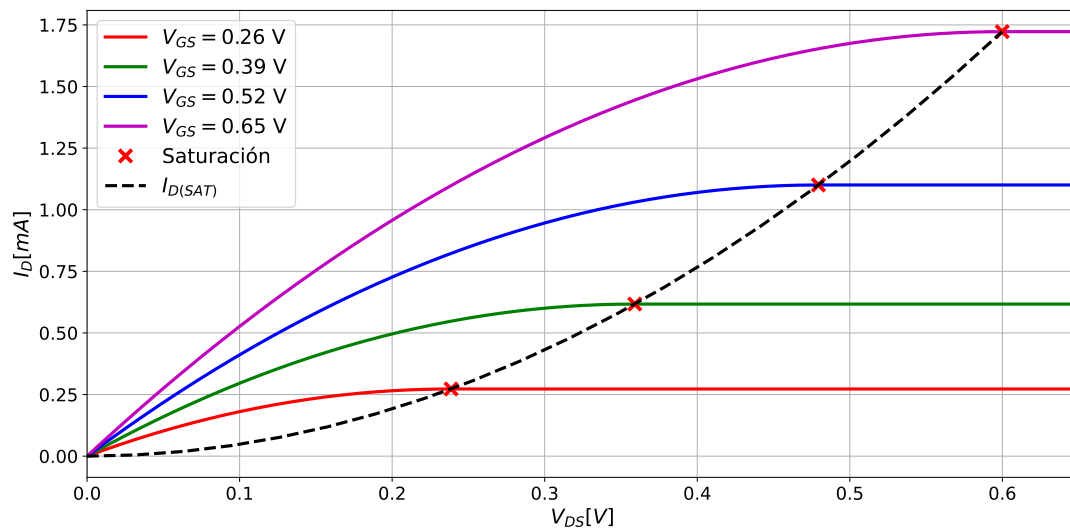


Figura 10: Curva de salida del transistor de canal corto.

La Figura 10 muestra la curva de salida del transistor de canal corto según el modelo cuadrático. Los valores de V_{GS} utilizados fueron $V_{GS} = \{0,26 \text{ V}; 0,39 \text{ V}; 0,52 \text{ V}; 0,65 \text{ V}\}$, mientras que para V_{DS} se utilizó un vector de 300 puntos con valores entre 0 y V_{DD} . También se incluye la curva de saturación $I_{D(sat)}$ vs. $V_{DS(sat)}$ en línea punteada, así como los puntos en donde esta se produce marcados con cruces rojas.

Comparando la Figura 10 con la curva de salida para el transistor de canal largo (Figura 3) puede observarse que la corriente de *drain* disminuye aproximadamente 10 veces, lo que es esperable al reducir

las dimensiones de dispositivo. La Figura 10 se usará como base para analizar cada uno de los efectos de canal corto estudiados.

Modulación del largo del canal

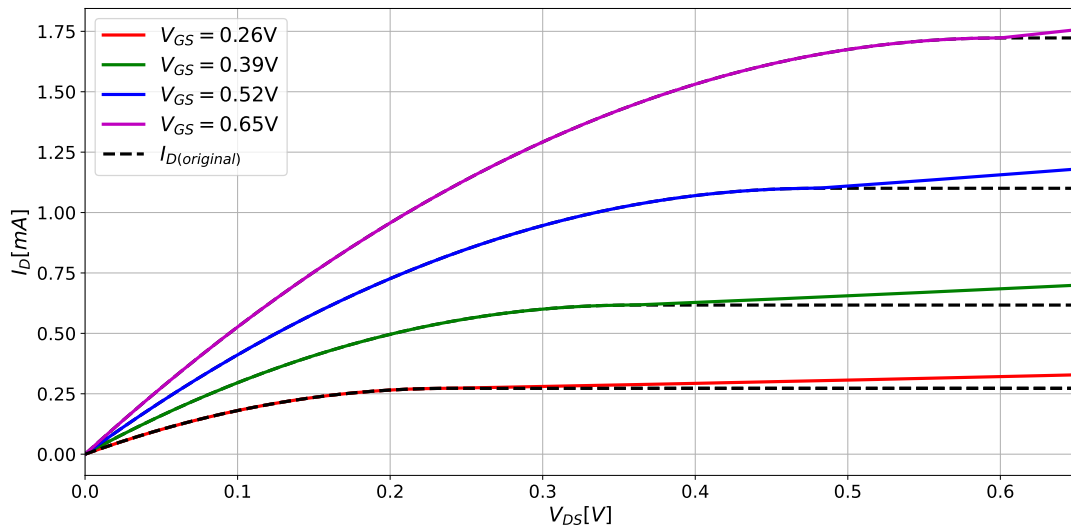


Figura 11: Curva de salida con E.M.L.C.

La Figura 11 muestra las curvas de salida teniendo en cuenta el E.M.L.C. y las curvas de salida originales en línea punteada negra. Se observa que dicho efecto solo influye en la región de saturación (que es lo que se asumió en la Subsección 3.2) y que a partir de dicho punto la corriente tiende a aumentar debido a que la longitud del canal disminuye. En tal caso, la corriente de saturación empieza a depender de V_{DS} y no depende solamente de V_{GS} .

D.I.B.L.

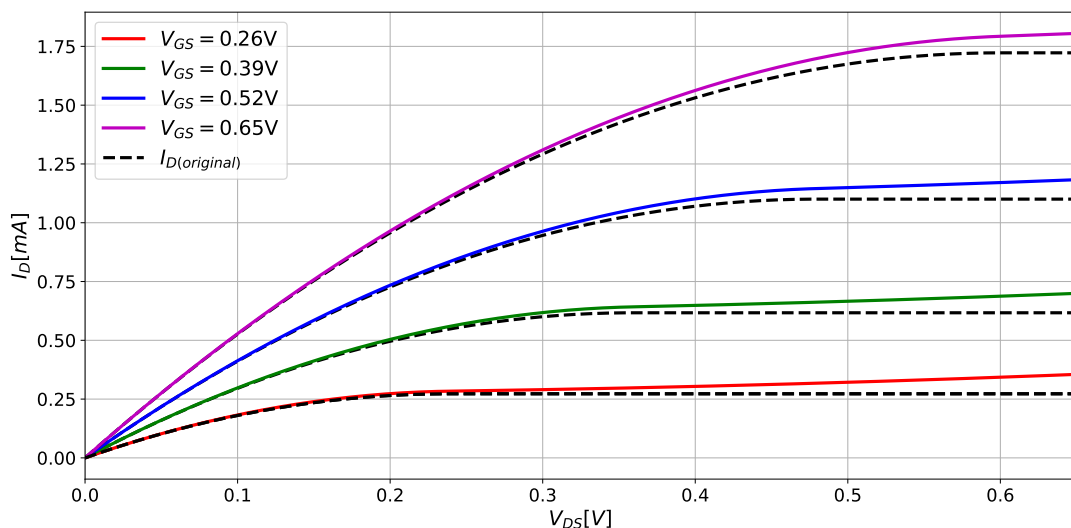


Figura 12: Curva de salida con efecto D.I.B.L.

La Figura 12 muestra las curvas de salida incluyendo el efecto D.I.B.L, así como las curvas de salida originales en línea negra punteada. Puede observarse que, a diferencia de el efecto de modulación del largo del canal, este efecto modifica la corriente para todos los valores de V_{DS} , aunque se hace más notorio para valores altos, como sugiere la Ecuación 23. Como se explicó en la Subsección 3.3, siempre

hay una corriente de D.I.B.L. paralela a la corriente de *drain* debido a que $V_{PT} < 0$, de forma tal que $I_{total} = I_D(V_{GS}, V_{DS}) + I_{D.I.B.L.}(V_{DS})$. Sin embargo, la simulación muestra que la corriente I_D domina el comportamiento del transistor para un amplio rango de valores. Por último, cabe mencionar que si se graficara el caso $V_{GS} = 0$, se observaría solo la corriente de D.I.B.L. y que el transistor no podría apagarse.

Degradación de la movilidad

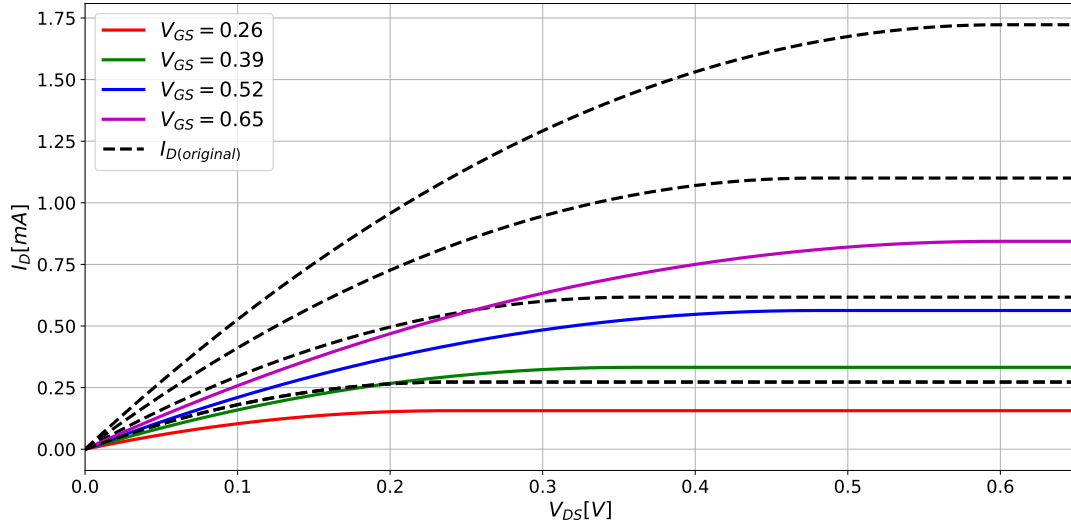


Figura 13: Curva de salida con degradación de la movilidad.

La Figura 13 muestra las curvas de salida incluyendo el efecto de degradación de la movilidad explicado en la Subsección 3.4, así como las curvas de salida originales de la Figura 10. Se observa una clara reducción de la corriente debida a este efecto. Los cálculos numéricos muestran que, en el mejor de los casos, la movilidad se reduce un 40 % (caso $V_{GS} = 0,26$ V), y hasta un 50 % en el peor (caso $V_{GS} = 0,65$ V). Esto es consistente con las ecuaciones 24 y 26, según las cuales cuanto mayor es V_{GS} mayor es el campo efectivo en el canal, lo que tiende a reducir la movilidad.

Saturación de la velocidad de arrastre

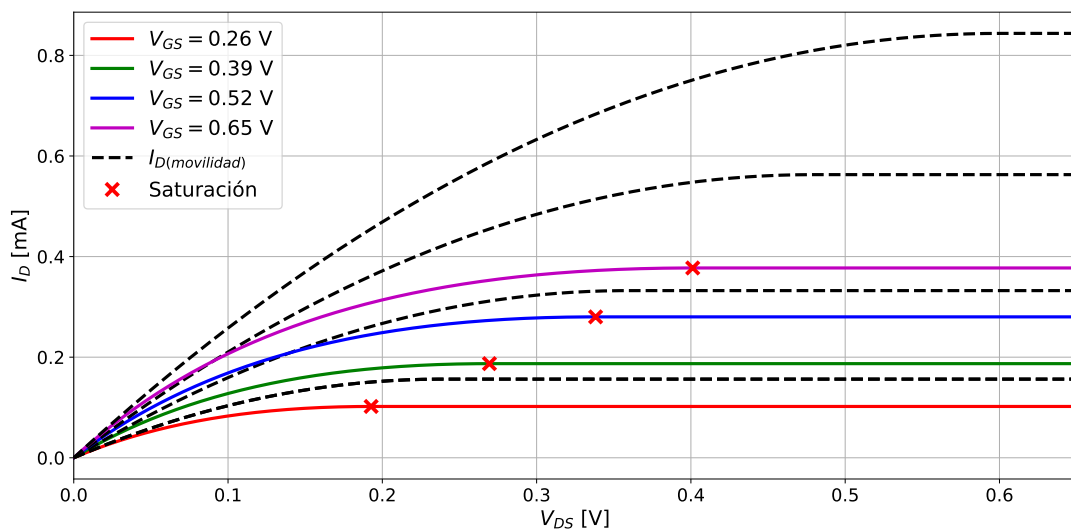


Figura 14: Curva de salida con saturación de la velocidad de arrastre.

La Figura 14 muestra el efecto que la saturación de la velocidad de arrastre tiene sobre la curva de salida. En línea punteada negra se muestra la corriente afectada por la degradación de la movilidad (es decir, la corriente de la Figura 13) y con cruces rojas se muestran los puntos $(I_{D(sat)}, V_{DS(sat)})$ dados por las ecuaciones 28 y 29. Se observa de esta manera que la saturación de la velocidad de arrastre contribuye a reducir aún más la corriente en el canal, ya que el campo dentro de este (V_{DS}/L) alcanza órdenes de magnitud similares a $\mathcal{E}_c$.

Combinación de todos los efectos

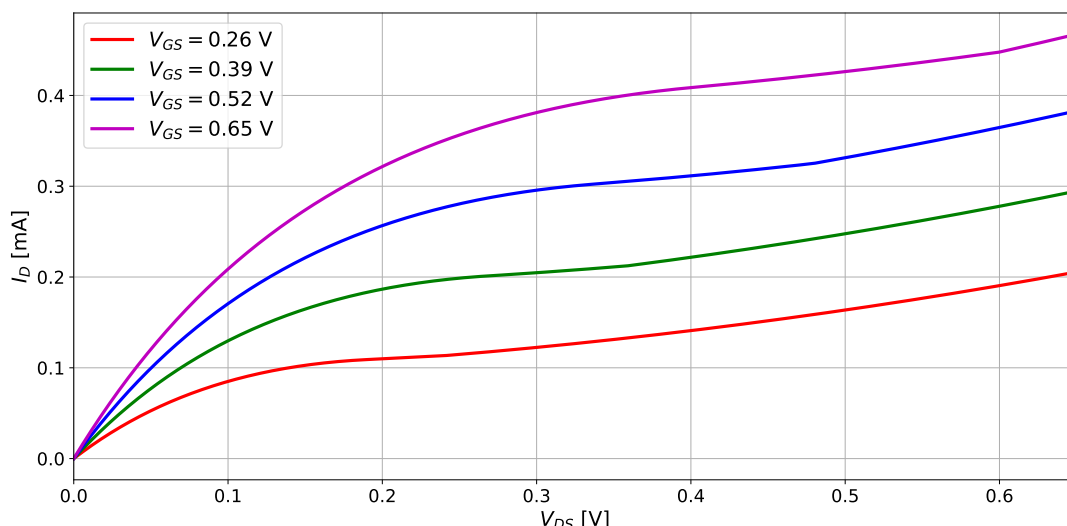


Figura 15: Curva de salida con todos los efectos

La Figura 15 muestra el resultado de superponer todos los efectos. No se incluyen las curvas de salida originales, ya que difieren de las de la Figura 15 en órdenes de magnitud, por lo que graficar ambas en una misma figura entorpecería la comprensión. Las corrientes de estas curvas se calcularon añadiendo el E.M.L.C. a las corrientes de la Figura 14 y sumando la corriente del efecto D.I.B.L., es decir

$$I_{final} = \frac{L}{L - \Delta L} I_{saturación} v_a + I_{D.I.B.L.} \quad (30)$$

A partir del análisis de esta figura y de las anteriores se puede determinar que los efectos que más impactan en las curvas de salida son la degradación de la movilidad y la saturación de la velocidad de arrastre. Esto se basa en el hecho de que estos dos efectos solos son responsables de reducir la corriente I_D en un 80 % en el peor de los casos y en un 60 % en el mejor. También puede observarse en la Figura 15 que estos 2 efectos combinados reducen tanto la corriente de salida que la corriente de D.I.B.L. tiene ahora un impacto claro, el cual era antes casi despreciable. Esto se puede apreciar en el crecimiento cuadrático para los valores más grandes de V_{DS} .

Análisis de calidad del transistor obtenido

La primera característica que se destaca del transistor obtenido es el bajo valor de la tensión de umbral V_T . En principio esto resulta deseable, ya que permitiría al transistor conmutar de forma rápida y precisa entre los estados 0 y 1 con pequeños valores de V_{GS} . Sin embargo, esto también tiene una contraparte, ya que el valor de V_{GS} debe ser controlado con mucha precisión para evitar que el transistor conduzca cuando no debe. Otras propiedades a destacar de la Tabla 3.3 incluyen el hecho de que $m \approx 1$ (lo que contribuye a disminuir la corriente de subumbral y por lo tanto la corriente de fuga) y que el valor de C_{ox} sea chico, ya que esto permite acortar los transitorios y por lo tanto el tiempo de conmutación.

Otra de las características deseables de un transistor, según se explicó en la Subsección 3.6 es una baja resistencia de salida. Para calcularla se usará la inversa de la conductancia de salida, es decir $r_o = g_o^{-1} = \left(\frac{\partial I_D}{\partial V_{DS}} \right)^{-1}$, por lo que se puede calcular a partir de la curva de salida. Se esperaría entonces que un buen transistor tuviera una pendiente elevada en la gráfica I_D vs. V_{DS} en la región de triodo, lo que equivaldría a tener una gran g_o o una r_o pequeña.

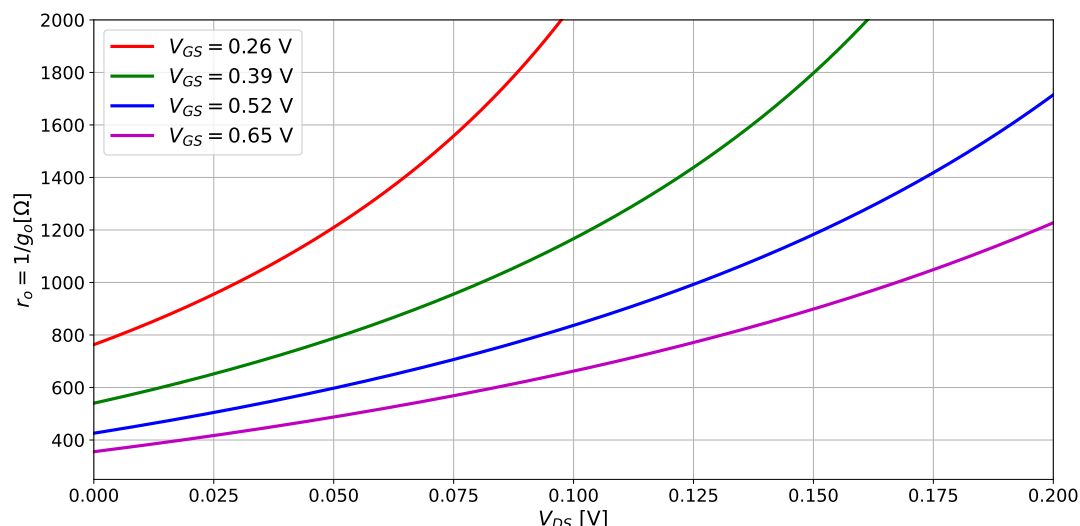


Figura 16: Resistencia de salida

Se observa en la Figura 16 que la resistencia de salida nunca es menor que 300 Ω, lo que constituye una característica altamente indeseable. La principal causa de este fenómeno es la rápida saturación de la corriente por la degradación de la movilidad y la saturación de la velocidad de arrastre, efectos que no permiten tener pendientes elevadas en la Figura 15, y que por lo tanto perjudican a la resistencia de salida.

Por último, el efecto D.I.B.L. hace que el transistor presente un comportamiento incompatible con el de una llave. Como se explicó en la Subsección 3.3, la tensión V_{PT} es negativa, lo que quiere decir que incluso para el caso $V_{DS} = 0$ las zonas de vaciamiento de las junturas SB y DB atraviesan el canal, permitiendo así que circule corriente al aplicar una tensión V_{DS} aún si la tensión V_{GS} fuera nula. En otras palabras, el transistor conduce cuando no debe, lo que hace imposible cualquier aplicación digital que se le quiera dar además de aumentar el consumo de energía en reposo.

4. Conclusiones

Se obtuvo la relación $\psi_{s,d}$ vs. V_{DS} para distintos valores de V_{GS} a través del método Newton-Raphson según se explicó en la Subsección 2.2. Se obtuvo que $\psi_{s,d}$ crece linealmente con V_{DS} siguiendo una relación similar a $\psi_{s,d} = V_{DS} - 2\psi_B$ hasta el punto de saturación, a partir del cual $\psi_{s,d}$ mantiene un valor constante. Esto se debe a que, para una tensión V_{DS} mayor, el potencial en el terminal del *drain* es mayor, lo que tiende a deshacer la carga de inversión en ese extremo del canal y producir el estrangulamiento o *pinch-off*.

Los 3 modelos comparados reproducen el comportamiento esperado de un MOSFET. Al compararlos, se obtuvo que los modelos numérico y cuadrático presentan resultados similares en todas las regiones de operación del dispositivo, por lo que este último resulta una aproximación útil del primero. En cambio, el modelo clásico presenta evidentes diferencias ya que, por un lado, $m \approx 1$ resulta una aproximación poco precisa y, por otro, no contempla la corriente en la región subumbral.

La guía de escalado a campo constante resulta una herramienta útil para diseñar un transistor de canal corto a partir de uno de canal largo. Sin embargo, no asegura que los efectos de canal corto no distorsionen el comportamiento del circuito. La causa de esto puede atribuirse a que el transistor de canal largo estudiado en la Parte I ya presentaba efectos no ideales, como la saturación de la velocidad de arrastre.

Se estudiaron varios efectos de canal corto, entre los cuales se destacan la degradación de la movilidad y la saturación de la velocidad de arrastre. Según lo explicado en las secciones 3.4 y 3.5 estos son los efectos que mayor impacto tienen en la curva de salida y más la apartan del modelo cuadrático dado por la Ecuación 17. A estos efectos solos puede atribuirse una reducción de hasta un 80 % en la corriente de salida.

El transistor obtenido resulta inviable para una aplicación digital en un dispositivo V.L.S.I. dada la elevada resistencia de salida que presenta, así como su incapacidad para interrumpir el flujo de corriente debido al efecto D.I.B.L.

A. Apéndice: Código Parte I

```

import numpy as np
from scipy.optimize import newton
import matplotlib.pyplot as plt
import os

os.makedirs("parteI", exist_ok=True)
#Crea la carpeta parteI si no existe todavía
os.makedirs("parteI/salida", exist_ok=True)
os.makedirs("parteI/transfereencia", exist_ok=True)

#Configuro el tamaño de todas las figuras (ancho, alto)
plt.rcParams["figure.figsize"] = (12, 6)

# Set font size for axis labels (e.g., "X-axis", "Y-axis")
plt.rcParams['axes.labelsize'] = 16
# Set font size for tick labels (the numbers on the axes)
plt.rcParams['xtick.labelsize'] = 14
plt.rcParams['ytick.labelsize'] = 14

plt.rcParams['legend.fontsize'] = 14
# Set global line thickness
plt.rcParams['lines.linewidth'] = 2.5

#####
#Constantes
#####

q = 1.602e-19
Vth = 25.875e-3 #en V
Na = 6e15 #en cm-3
ni = 1e10 #en cm-3

eps_r_si = 11.7
eps_r_ox = 3.9
eps_0 = 8.854e-14 #en F/cm
eps_si = eps_r_si * eps_0
eps_ox = eps_r_ox * eps_0

tox = 35e-7 #en cm
Cox = eps_ox/tox #en F/cm2

VFB = -0.87 #en V

VDD = 6.5 #en V

psi_B = -344.259e-3 #en V

mu_n = 1245.1 #en cm2/V.s

W = 16e-4 #en cm
L = 1.5e-4 #en cm

gamma = np.sqrt(2 * q * Na * eps_si)/Cox #en V(1/2)
m = 1 + gamma/(2 * np.sqrt(-2 * psi_B))

VT = VFB - 2 * psi_B + gamma * np.sqrt(-2*psi_B) #en V

```

```

#funciones
def Qs2(psi, VDS): #Carga al cuadrado
    return (2*q*Vth*Na*eps_si*(psi/Vth + (ni**2/Na**2)*np.exp((psi-VDS)/Vth)))

def dQs2_dpsi(psi, VDS): #derivada de la carga al cuadrado
    return (2*q*Na*eps_si*(1 + (ni**2/Na**2)*np.exp((psi-VDS)/Vth)))

def f(psi, VDS, VGS):
    return Qs2(psi, VDS) - Cox**2*(VGS - VFB - psi)**2

def df(psi, VDS, VGS):
    return dQs2_dpsi(psi, VDS) + 2*Cox**2*(VGS - VFB - psi)

VGS_vec = np.array([0.4, 0.6, 0.8, 1]) * VDD

VDS_vec = np.linspace(0, VDD, 300)

psi_s_d_vec = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VD

psi0 = 0.0 # estimación inicial

for k, VGS in enumerate(VGS_vec):
    for i, VDS in enumerate(VDS_vec):
        psi = newton(
            func = f,
            x0 = psi0,
            fprime = df,
            args = (VDS, VGS,),
            tol = 1e-10,
            maxiter = 500
        )

        psi_s_d_vec[k, i] = psi
        psi0 = psi

for k, VGS in enumerate(VGS_vec):
    plt.plot(VDS_vec, psi_s_d_vec[k, :], label=f"VGS={VGS:.1f} V")

plt.plot(VDS_vec, VDS_vec - 2 * psi_B, 'k--', label = r'\mathcal{V}_{DS} - 2\psi_B')
plt.xlabel(r'\mathcal{V}_{DS} [V]\mathcal{L}')
plt.xlim(0, VDD)
plt.ylabel(r'\mathcal{L}\psi_{s,d} [V]\mathcal{L}')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("parteI/salida/psi_s_d.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

def INTEGRAL(psi, VGS): #Integral de la ecuación 13.21. EL RESULTADO ESTÁ EN AMPERE
    return mu_n * W/L * (Cox * (VGS - VFB + Vth) * psi
        - 0.5 * Cox*psi**2 -
        2/3 * np.sqrt(2 * eps_si * q * Na) * psi**(3/2) + Vth * np.sqrt(2 * eps_si *

```

```

ID_num = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS
#cálculo numérico
for k, VGS in enumerate(VGS_vec):
    for i, VDS in enumerate(VDS_vec):
        ID_num[k, i] = INTEGRAL(psi_s_d_vec[k, i], VGS) - INTEGRAL(psi_s_s[k], VGS)

for k, VGS in enumerate(VGS_vec):
    plt.plot(VDS_vec, ID_num[k, :] * 1e3, label=f"VGS={VGS:.1f} V")
plt.xlabel(r" $V_{DS}$  [V]")
plt.xlim(0, VDD)
plt.ylabel(r" $I_{D}$  [mA]")
plt.grid(True)
plt.legend()
plt.show()

#Para cada caso de VGS calculo un VDS(sat) a partir de la intersección
#de la recta VDS - 2*psi_B y la curva psi_s,d.
#Así obtengo un valor de VDS(sat) y un valor de psi_s,d(sat)
#Con el valor psi_s,d(sat) calculo la corriente con la integral

N = 20
VGS_vec2 = np.linspace(VT, VDD, N)

VDS_vec2 = np.linspace(0, VDD, 300)

psi_s_d_vec2 = np.zeros((len(VGS_vec2), len(VDS_vec2))) #Una fila por cada VGS, una columna por cada
psi0 = 0.0 # estimación inicial

for k, VGS in enumerate(VGS_vec2):
    for i, VDS in enumerate(VDS_vec2):
        psi = newton(
            func = f,
            x0 = psi0,
            fprime = df,
            args = (VDS, VGS,),
            tol = 1e-10,
            maxiter = 500
        )

        psi_s_d_vec2[k, i] = psi
        psi0 = psi

recta = VDS_vec2 - 2 * psi_B #1 fila x 300 col

dif = np.abs(psi_s_d_vec2 - recta) #N fil x 300 col
#A cada fila de psi_s_d_vec2 le resto la recta VDS - 2 * psi_B. Tomo el valor absoluto

idx_VDS_SAT = np.argmin(dif, axis = 1) #N fil
#busco dónde el valor absoluto de la resta se hace mínimo. Uno por cada valor de VGS
VDSsat_parab_num = VDS_vec2[idx_VDS_SAT] #N fil
#En esos índices se produce VDS_SAT

psi_s_d_sat = np.zeros_like(VGS_vec2) #N fil

```

```

for i in range(len(VGS_vec2)):
    psi_s_d_sat[i] = psi_s_d_vec2[i, idx_VDS_SAT[i]]

psi_s_s2 = psi_s_d_vec2[:, 0]
#Con psi_s_d_sat y psi_s_s2 calculo IDSat

IDSat_parab_num = np.zeros_like(VDSsat_parab_num)

for i in range(len(VDSsat_parab_num)):
    IDSat_parab_num[i] = INTEGRAL(psi_s_d_sat[i], VGS_vec2[i]) - INTEGRAL(psi_s_s2[i], VGS_vec2[i])

plt.plot(VDSsat_parab_num, IDSat_parab_num * 1e3, 'rx')
plt.grid(True)
plt.xlabel(r" $V_{DS}$  [V] $\mu$ ")
plt.xlim(0, VDD)
plt.ylabel(r" $I_{D(SAT)}$  [mA] $\mu$ ")
plt.show()

#Calculo VDSsat e IDSat para cada uno de los cuatro casos de VGS
dif = np.abs(psi_s_d_vec - recta)
idx_VDS_SAT = np.argmin(dif, axis = 1)
VDSsat_num = VDS_vec[idx_VDS_SAT]
psi_s_d_sat = np.zeros_like(VDSsat_num)
for i in range(len(VDSsat_num)):
    psi_s_d_sat[i] = psi_s_d_vec[i, idx_VDS_SAT[i]]
psi_s_s2 = psi_s_d_vec[:, 0]
IDSat_num = np.zeros_like(VDSsat_num)
for i in range(len(VDSsat_num)):
    IDSat_num[i] = INTEGRAL(psi_s_d_sat[i], VGS_vec[i]) - INTEGRAL(psi_s_s2[i], VGS_vec[i])

for k, VGS in enumerate(VGS_vec):
    plt.plot(VDS_vec, ID_num[k, :] * 1e3, label=f"VGS={VGS:.1f} V")

plt.plot(VDSsat_parab_num, IDSat_parab_num * 1e3, 'k--', label = r" $I_{D(SAT)}$  $\mu$ ")

plt.plot(VDSsat_num, IDSat_num * 1000, 'rx')
plt.xlabel(r" $V_{DS}$  [V] $\mu$ ")
plt.xlim(0, VDD)
plt.ylabel(r" $I_{D}$  [mA] $\mu$ ")
plt.grid(True)
plt.legend()
plt.savefig("parteI/salida/num.pdf", format = "pdf")
plt.show()

ID_cuad = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS

ID_cuad = mu_n * Cox * W/L * (VGS_vec[:, None] - VT - (m/2) * VDS_vec) * VDS_vec

VDSsat_cuad = 1/m * (VGS_vec - VT) #un VDS_Sat por cada caso de VGS

IDSat_cuad = mu_n * Cox * W/L * 1/(2 * m) * (VGS_vec - VT)**2
#VDSsat_cuad e IDSat_cuad guardan los puntos de saturación para los 4 casos de VGS

for k, VGS in enumerate(VGS_vec):
    idx = VDS_vec > VDSsat_cuad[k]
    ID_cuad[k, idx] = IDSat_cuad[k]
#fuerzo a que a partir de VDS_sat, la corriente se mantenga constante

```

```

#Además, quiero ver ID_sat vs VDS_sat
VDSsat_parab_cuad = 1/m * (VGS_vec2 - VT)
IDSat_parab_cuad = mu_n * Cox * W/L * 1/(2 * m) * (VGS_vec2 - VT)**2

for k, VGS in enumerate(VGS_vec):
    plt.plot(VDS_vec, ID_cuad[k, :] * 1e3, label=f"VGS={VGS:.1f} V")

plt.plot(VDSsat_cuad, IDsat_cuad * 1e3, 'rx')
plt.plot(VDSsat_parab_cuad, IDsat_parab_cuad * 1e3, 'k--', label = r'$I_{D(SAT)}$')
plt.xlabel(r"$V_{DS}$ [V]")
plt.xlim(0, VDD)
plt.ylabel(r"$I_{D}$ [mA]")
plt.grid(True)
plt.legend()
plt.savefig("parteI/salida/cuad.pdf", format = "pdf")
plt.show()

ID_cla = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS

ID_cla = mu_n * Cox * W/L * (VGS_vec[:, None] - VT - (1/2) * VDS_vec) * VDS_vec

VDSsat_cla = (VGS_vec - VT) #un VDS_Sat por cada caso de VGS

IDSat_cla = mu_n * Cox * W/L * 1/2 * (VGS_vec - VT)**2
#VDSsat_cla e IDsat_cla guardan los puntos de saturación para los 4 casos de VGS

for k, VGS in enumerate(VGS_vec):
    idx = VDS_vec > VDSsat_cla[k]
    ID_cla[k, idx] = IDsat_cla[k]
#fuerzo a que a partir de VDS_sat, la corriente se mantenga constante

#Además, quiero ver ID_sat vs VDS_sat
VDSsat_parab_cla = (VGS_vec2 - VT)
IDSat_parab_cla = mu_n * Cox * W/L * 1/2 * (VGS_vec2 - VT)**2

for k, VGS in enumerate(VGS_vec):
    plt.plot(VDS_vec, ID_cla[k, :] * 1e3, label=f"VGS={VGS:.1f} V")

plt.plot(VDSsat_cla, IDsat_cla * 1e3, 'rx')
plt.plot(VDSsat_parab_cla, IDsat_parab_cla * 1e3, 'k--', label = r'$I_{D(SAT)}$')
plt.xlabel(r"$V_{DS}$ [V]")
plt.xlim(0, VDD)
plt.ylabel(r"$I_{D}$ [mA]")
plt.grid(True)
plt.legend()
plt.savefig("parteI/salida/cla.pdf", format = "pdf")
plt.show()

plt.plot(VDS_vec, ID_num[2, :] * 1e3, 'b-', label = "Numérico") #Resultado numérico
plt.plot(VDS_vec, ID_cuad[2, :] * 1e3, 'r-', label = "Cuadrático") #Resultado con aproximación cuadrática
plt.plot(VDS_vec, ID_cla[2, :] * 1e3, 'g-', label = "Clásico") #Resultado clásico

idx_num = IDsat_parab_num < IDsat_num[2]
idx_cuad = IDsat_parab_cuad < IDsat_cuad[2]
idx_cla = IDsat_parab_cla < IDsat_cla[2]

plt.plot(VDSsat_parab_num[idx_num], IDsat_parab_num[idx_num] * 1e3, 'b--') #Resultado numérico
plt.plot(VDSsat_parab_cuad[idx_cuad], IDsat_parab_cuad[idx_cuad] * 1e3, 'r--') #Resultado con aproximación cuadrática

```

```

plt.plot(VDSsat_parab_cla[idx_cla], IDSat_parab_cla[idx_cla] * 1e3, 'g--') #Resultado clásico

plt.title(r'Curvas de salida para  $I_{V_{GS}} = 0,8 \cdot V_{DD} = I' f' \{0.8 * VDD\} V'$ )
plt.xlabel(r" $I_{V_{DS}} [V]$ ")
plt.xlim(0, VDD)
plt.ylabel(r" $I_{ID} [mA]$ ")
plt.grid(True)
plt.legend()
plt.savefig("parteI/salida/ID_comparacion.pdf", format = "pdf")
plt.show()

%reset -f

import numpy as np
from scipy.optimize import newton
import matplotlib.pyplot as plt

#####
#Constantes
#####

q = 1.602e-19
Vth = 25.875e-3 #en V
Na = 6e15 #en cm^-3
ni = 1e10 #en cm^-3

eps_r_si = 11.7
eps_r_ox = 3.9
eps_0 = 8.854e-14 #en F/cm
eps_si = eps_r_si * eps_0
eps_ox = eps_r_ox * eps_0

tox = 35e-7 #en cm
Cox = eps_ox/tox #en F/cm^2

VFB = -0.87 #en V

VDD = 6.5 #en V

psi_B = -344.259e-3 #en V

mu_n = 1245.1 #en cm^2/V.s

W = 16e-4 #en cm
L = 1.5e-4 #en cm

gamma = np.sqrt(2 * q * Na * eps_si)/Cox #en V^(1/2)
m = 1 + gamma/(2 * np.sqrt(-2 * psi_B))

VT = VFB - 2 * psi_B + gamma * np.sqrt(-2*psi_B) #en V

#Como siempre tengo un gráfico en escala lineal y al lado su versión en escala semilog,
#para ponerlos uno al lado del otro conviene que sean cuadrados
plt.rcParams["figure.figsize"] = (6, 6)

#funciones
def Qs2(psi, VDS): #Carga al cuadrado
    return (2*q*Vth*Na*eps_si*(psi/Vth + (ni**2/Na**2)*np.exp((psi-VDS)/Vth)))

```

```

def dQs2_dpsi(psi, VDS): #derivada de la carga al cuadrado
    return (2*q*Na*eps_si* (1 + (ni**2/Na**2)*np.exp((psi-VDS)/Vth)))

def f(psi, VDS, VGS):
    return Qs2(psi, VDS) - Cox**2*(VGS - VFB - psi)**2

def df(psi, VDS, VGS):
    return dQs2_dpsi(psi, VDS) + 2*Cox**2*(VGS - VFB - psi)

def INTEGRAL(psi, VGS): #Integral de la ecuación 13.21. EL RESULTADO ESTÁ EN AMPERE
    return mu_n * W/L * (Cox * (VGS - VFB + Vth) * psi
        - 0.5 * Cox*psi**2 -
        2/3 * np.sqrt(2 * eps_si * q * Na) * psi**(3/2) + Vth * np.sqrt(2 * eps_si *

VDS_vec = np.array([Vth, 0.1 * VDD, 0.5 * VDD, VDD])
N = 300
VGS_vec = np.linspace(0, VDD, N)

psi_s_d_vec = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS

psi0 = 0.0 # estimación inicial

#Determino psi_s_d para cada caso de VDS y VGS
for k, VGS in enumerate(VGS_vec):
    for i, VDS in enumerate(VDS_vec):
        psi = newton(func = f, x0 = psi0, fprime = df, args = (VDS, VGS,), tol = 1e-10, maxiter = 500)
        psi_s_d_vec[k, i] = psi
        psi0 = psi

#Ahora, para cada caso de VGS tengo que calcular psi_s_s (ocurre para la condición VDS = 0)
psi_s_s = np.zeros_like(VGS_vec) #un psi_s_s para cada caso de VGS

psi_s_s0 = 0
for k, VGS in enumerate(VGS_vec):
    psi = newton(func = f, x0 = psi_s_s0, fprime = df, args = (0, VGS,), tol = 1e-10, maxiter = 500)
    psi_s_s[k] = psi

ID_num = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS
#cálculo numérico
for i, VDS in enumerate(VDS_vec):
    for k, VGS in enumerate(VGS_vec):
        ID_num[k, i] = INTEGRAL(psi_s_d_vec[k, i], VGS) - INTEGRAL(psi_s_s[k], VGS)

color_arr = ['black', 'orange', 'green', 'red']
line_arr = ['-', '--', '-.', ':']

for k, VDS in enumerate(VDS_vec):
    plt.plot(VGS_vec, ID_num[:, k] * 1e3, color = color_arr[k], label = r' $\mathcal{I}_{V_{DS}} = \mathcal{I}' f' \{VDS: .3f$ '

plt.xlabel(r' $\mathcal{V}_{GS} [V]\mathcal{I}$ ')
plt.xlim(0, VDD)
plt.ylabel(r' $\mathcal{I}_{ID} [mA]\mathcal{I}$ ')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("parteI/transferencia/num.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

```

```

for k, VDS in enumerate(VDS_vec):
    plt.plot(VGS_vec, ID_num[:, k], color = color_arr[k], linestyle = line_arr[k], label=f "VDS={VDS}

plt.xlabel(r "\mathcal{V}_{GS} [V]\mathcal{L}")
plt.xlim(0, 1)
plt.semilogy()
plt.ylabel(r "\mathcal{I}_D [A]\mathcal{L}")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("parteI/transferencia/num_log.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

for i, VDS in enumerate(VDS_vec):
    plt.plot(VGS_vec, psi_s_d_vec[:, i], label = f 'VDS = {VDS} ')

plt.xlabel(r "\mathcal{V}_{GS} [V]\mathcal{L}")
plt.xlim(0, VDD)
plt.ylabel(r "\mathcal{I}_{psi_{s,d}} [V]\mathcal{L}")
plt.grid(True)
plt.legend()

ID_cuad = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS

VDSsat = 1/m * (VGS_vec - VT)

#Por cada punto de la grilla bidimensional de (VGS, VDS) aplico la ecuación que corresponda
for i, VDS in enumerate(VDS_vec):
    for k, VGS in enumerate(VGS_vec):
        if VGS < VT:
            ID_cuad[k, i] = 0
        elif VDS < VDSsat[k]:
            ID_cuad[k, i] = mu_n * Cox * W/L * (VGS - VT - (m/2) * VDS) * VDS
        elif VDS >= VDSsat[k]:
            ID_cuad[k, i] = mu_n * Cox * W/L * 1/(2*m) * (VGS - VT)**2

ID_subumbral = np.zeros((len(VGS_vec), len(VDS_vec)))

for i, VDS in enumerate(VDS_vec):
    ID_subumbral[:, i] = mu_n * Cox * W/L * (m - 1) * Vth**2 * np.exp((VGS_vec - VT)/(m * Vth)) * (1 -
    idx_umbral = np.argmin(np.abs(VGS_vec - VT))
    ID_subumbral[VGS_vec > VT, i] = ID_subumbral[idx_umbral, i]
    #Para que la discontinuidad no se note, a partir del punto en el cual VGS > VT sumo la misma corri
    #Para los valores VGS > VT, la misma se vuelve despreciable y no afecta al resto del gráfico.
    #Esto es válido incluso para el caso VDS = Vth

ID_cuad = ID_cuad + ID_subumbral

for k, VDS in enumerate(VDS_vec):
    plt.plot(VGS_vec, ID_cuad[:, k] * 1000, color = color_arr[k], label = r "\mathcal{V}_{DS} =\mathcal{L} f '{VDS:.3}

plt.grid()
plt.xlabel(r "\mathcal{V}_{GS} [V]\mathcal{L}")
plt.ylabel(r "\mathcal{I}_D [mA]\mathcal{L}")
plt.legend()
plt.tight_layout()
plt.savefig("parteI/transferencia/cuad.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

```



```

for k, VDS in enumerate(VDS_vec):
    plt.plot(VGS_vec, ID_cuad[:, k], color = color_arr[k], linestyle = line_arr[k], label = r'$V_{DS}$')

plt.grid()
plt.semilogy()
plt.xlabel(r'$V_{GS}$ [V]')
plt.xlim([0, 1])
plt.ylabel(r'$I_D$ [A]')
plt.legend()
plt.tight_layout()
plt.savefig('parteI/transferencia/cuad_log.pdf', format = "pdf", bbox_inches = 'tight')
plt.show()

ID_cla = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS

VDSsat = (VGS_vec - VT)

#Por cada punto de la grilla bidimensional de (VGS, VDS) aplico la ecuación que corresponda
for k, VGS in enumerate(VGS_vec):
    for i, VDS in enumerate(VDS_vec):
        if VGS < VT:
            ID_cla[k, i] = 0
        elif VDS < VDSsat[k]:
            ID_cla[k, i] = mu_n * Cox * W/L * (VGS - VT - (1/2) * VDS) * VDS
        elif VDS >= VDSsat[k]:
            ID_cla[k, i] = mu_n * Cox * W/L * 1/2 * (VGS - VT)**2

for k, VDS in enumerate(VDS_vec):
    plt.plot(VGS_vec, ID_cla[:, k] * 1000, color = color_arr[k], label = r'$V_{DS}$ = %f V' % VDS)

plt.grid()
plt.xlabel(r'$V_{GS}$ [V]')
plt.xlim(0, VDD)
plt.ylabel(r'$I_D$ [mA]')
plt.legend()
plt.tight_layout()
plt.savefig('parteI/transferencia/cla.pdf', format = "pdf", bbox_inches = 'tight')
plt.show()

for k, VDS in enumerate(VDS_vec):
    plt.plot(VGS_vec, ID_cla[:, k], color = color_arr[k], linestyle = line_arr[k], label = r'$V_{DS}$')

plt.grid()
plt.xlabel(r'$V_{GS}$ [V]')
plt.xlim(0, 1)
plt.ylabel(r'$I_D$ [A]')
plt.semilogy()
plt.legend()
plt.tight_layout()
plt.savefig('parteI/transferencia/cla_log.pdf', format = "pdf", bbox_inches = 'tight')
plt.show()

k = 2
plt.plot(VGS_vec, ID_num[:, k] * 1000, 'b-', label = 'Numérica')
plt.plot(VGS_vec, ID_cuad[:, k] * 1000, 'r--', label = 'Cuadrática')
plt.plot(VGS_vec, ID_cla[:, k] * 1000, 'k-', label = 'Clásica')

```

```
plt.title(r'Curva de transferencia para  $\ell V_{DS} = 0,5 \cdot V_{DD} = \ell f' \{0.5 * VDD\} V'$ )

plt.grid()
plt.xlabel(r' $\ell V_{GS} [V]\ell'$ ')
plt.ylabel(r' $\ell I_D [mA]\ell'$ ')
plt.xlim(0, VDD)
plt.legend()
plt.tight_layout()
plt.savefig('parteI/transferencia/comp.pdf', format = "pdf", bbox_inches = 'tight')
plt.show()

k = 2
plt.plot(VGS_vec, ID_num[:, k], 'b-', label = 'Numérica')
plt.plot(VGS_vec, ID_cuad[:, k], 'r--', label = 'Cuadrática')
plt.plot(VGS_vec, ID_cla[:, k], 'k-.', label = 'Clásica')

plt.title(r'Curva de transferencia para  $\ell V_{DS} = 0,5 \cdot V_{DD} = \ell f' \{0.5 * VDD\} V'$ )

plt.grid()
plt.xlabel(r' $\ell V_{GS} [V]\ell'$ ')
plt.ylabel(r' $\ell I_D [A]\ell'$ ')
plt.semilogy()
plt.xlim(0, 1)
plt.legend()
plt.tight_layout()
plt.savefig('parteI/transferencia/comp_log.pdf', format = "pdf", bbox_inches = 'tight')
plt.show()
```

B. Apéndice: Código Parte II

```
import numpy as np
from scipy.optimize import newton
import matplotlib.pyplot as plt
import os

os.makedirs("parteII", exist_ok=True)
#Crea la carpeta parteII si no existe todavía

#Configuro el tamaño de todas las figuras (ancho, alto)
plt.rcParams["figure.figsize"] = (12, 6)

#Tamaño del texto de las etiquetas
plt.rcParams['axes.labelsize'] = 16
#tamaño del texto de los ejes
plt.rcParams['xtick.labelsize'] = 14
plt.rcParams['ytick.labelsize'] = 14
#tamaño del texto de la leyenda
plt.rcParams['legend.fontsize'] = 16
#Ancho de las líneas
plt.rcParams['lines.linewidth'] = 2.5
#tamaño de los marcadores
plt.rcParams['lines.markersize'] = 10
plt.rcParams['lines.markeredgewidth'] = 3

#####
#Constantes
#####
```

```

q = 1.602e-19
Vth = 25.875e-3 #en V
Na = 60e15 #en cm^-3
ni = 1e10 #en cm^-3

eps_r_si = 11.7
eps_r_ox = 3.9
eps_0 = 8.854e-14 #en F/cm
eps_si = eps_r_si * eps_0
eps_ox = eps_r_ox * eps_0

tox = 3.5e-7 #en cm
Cox = eps_ox/tox #en F/cm^2

psi_B = - Vth * np.log(Na/ni) #en V
phi_poly = 4.040 #en V
phi_si = 4.01 + 1.12/2 - psi_B

VFB = phi_poly - phi_si #en V

VDD = 0.65 #en V

mu_n = 842.7 #en cm^2/V.s

W = 1.6e-4 #en cm
L = 0.15e-4 #en cm
rj = 42e-7 #en cm

gamma = np.sqrt(2 * q * Na * eps_si)/Cox #en V^(1/2)
m = 1 + gamma/(2 * np.sqrt(-2 * psi_B))

VT = VFB - 2 * psi_B + gamma * np.sqrt(-2*psi_B)#en V

#Ejes
fmt_array = ['r-', 'g-', 'b-', 'm-']
VGS_vec = np.array([0.4, 0.6, 0.8, 1]) * VDD
VDS_vec = np.linspace(0, VDD, 5000)

#VGS para la parábola de saturación
VGS_sat_parab = np.linspace(VT, VDD, 20)

ID_cuad = np.zeros((len(VGS_vec), len(VDS_vec))) #Una fila por cada VGS, una columna por cada VDS

ID_cuad = mu_n * Cox * W/L * (VGS_vec[:, None] - VT - (m/2) * VDS_vec) * VDS_vec

VDS_sat = 1/m * (VGS_vec - VT)
ID_sat = mu_n * Cox * W/L * 1/(2 * m) * (VGS_vec - VT)**2
#Cada uno de los VDS_sat e ID_sat para los 4 casos de VGS

for k, VGS in enumerate(VGS_vec):
    idx = VDS_vec > VDS_sat[k] # = VDS_sat para cada uno de los 4 casos de VGS
    ID_cuad[k, idx] = ID_sat[k]
#fuerzo a que a partir de VDS_sat, la corriente se mantenga constante

#Además, quiero ver ID_sat vs VDS_sat
VDSsat_parab_cuad = 1/m * (VGS_sat_parab - VT)
IDSat_parab_cuad = mu_n * Cox * W/L * 1/(2 * m) * (VGS_sat_parab - VT)**2

```

```

for k, VGS in enumerate(VGS_vec):
    plt.plot(VDS_vec, ID_cuad[k, :] * 1e3, fmt_array[k], label = r'\mathcal{I}_{D(SAT)} = \mathcal{I}' + f'\{VGS: .2f\} V')

plt.plot(VDS_sat, ID_sat * 1e3, 'rx', label = 'Saturación')
plt.plot(VDSsat_parab_cuad, IDSat_parab_cuad * 1e3, 'k--', label = r'\mathcal{I}_{D(SAT)} = \mathcal{I}')
plt.xlabel(r'\mathcal{I}_{DS} [V]\mathcal{I}')
plt.xlim(0, VDD)
plt.ylabel(r'\mathcal{I}_{D} [mA]\mathcal{I}')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("parteII/salida.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

phi_bi = 0.56 - psi_B #Eg/2q - psi_B, tensión de contacto de la juntura DB en V
delta_L = np.zeros((len(VGS_vec), len(VDS_vec)), dtype = float)

for k, VGS in enumerate(VGS_vec):
    VDS_sat_k = VDS_sat[k]
    delta_L[k, :] = np.sqrt(2 * eps_si / (q * Na)) * (np.sqrt(phi_bi + VDS_vec) - np.sqrt(phi_bi + VDS_sat_k))
    #Para los puntos en donde no esté en saturación (para los puntos en la región lineal), delta_L debe ser 0
    idx = delta_L < 0
    delta_L[idx] = 0

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, delta_L[k, :] * 1e7, fmt_array[k], label = r'\mathcal{I}_{DS} = \mathcal{I}' + str(VGS_vec[k]) + ' V')

plt.grid()
plt.xlabel(r'\mathcal{I}_{DS} [V]\mathcal{I}')
plt.xlim([0, VDD])
plt.ylabel(r'\mathcal{I}_{Delta L} [nm]\mathcal{I}')
plt.ylim(0)
plt.legend()
plt.show()

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, delta_L[k, :]/L, fmt_array[k], label = r'\mathcal{I}_{DS} = \mathcal{I}' + str(VGS_vec[k]) + ' V')

plt.grid()
plt.xlabel(r'\mathcal{I}_{DS} [V]\mathcal{I}')
plt.xlim([0, VDD])
plt.ylabel(r'\mathcal{I}_{Delta L/L}\mathcal{I}')
plt.ylim(0)
plt.legend()
plt.show()

factor = L/(L - delta_L)
ID_EMLC = factor * ID_cuad

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, ID_cuad[k, :] * 1e3, 'k--')
    plt.plot(VDS_vec, ID_EMLC[k, :] * 1e3, fmt_array[k], label = r'\mathcal{I}_{DS} = \mathcal{I}' + str(VGS_vec[k]) + ' V')

plt.plot(VDS_vec, ID_cuad[0, :] * 1e3, 'k--', label = r'\mathcal{I}_{D(original)} = \mathcal{I}')
plt.grid()
plt.xlabel(r'\mathcal{I}_{DS} [V]\mathcal{I}')
plt.xlim([0, VDD])

```

```

plt.ylabel(r'$I_D$ [mA]')
plt.legend()
plt.tight_layout()
plt.savefig("parteII/EMLC.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

x_db0 = np.sqrt(2 * eps_si / (q * Na) * phi_bi) #ancho de la zona de vaciamiento en equilibrio (en cm)
print(f'x_db0 = {x_db0 * 1e7} nm')
print(f'L = {L * 1e7} nm')
VPT = q * Na / (2 * eps_si) * (L - x_db0)**2 - phi_bi
print(f'VPT = {VPT}')

J_dibl = np.zeros((len(VGS_vec), len(VDS_vec)), dtype = float)
for k, VGS in enumerate(VGS_vec):
    J_dibl[k, :] = 9/8 * eps_si * mu_n * 1/(L**3) * VDS_vec**2 #en A/cm2

ID_dibl = W * rj * J_dibl #en A

plt.plot(VDS_vec, ID_dibl[0, :] * 1e3, 'k--')

plt.grid()
plt.xlabel(r'$V_{DS}$ [V]')
plt.xlim([0, VDD])
plt.ylabel(r'$I_D$ [mA]')
plt.show()

ID_total = ID_cuad + ID_dibl

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, ID_cuad[k, :] * 1e3, 'k--')
    plt.plot(VDS_vec, ID_total[k, :] * 1e3, fmt_array[k], label = r'$V_{GS} = $' + str(VGS_vec[k]) + 'V')

plt.plot(VDS_vec, ID_cuad[0, :] * 1e3, 'k--', label = r'$I_D(original)$')
plt.grid()
plt.xlabel(r'$V_{DS}$ [V]')
plt.xlim([0, VDD])
plt.ylabel(r'$I_D$ [mA]')
plt.legend()
plt.tight_layout()
plt.savefig("parteII/DIBL.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

E_eff = eps_ox / (tox * eps_si) * (VT - VFB + 2 * psi_B) + eps_ox / (2 * tox * eps_si) * (VGS_vec - VT)

mu_0 = 975 #en cm^2/Vs
E0 = 20000 #en V/cm
alfa = -0.28
mu_eff = mu_0 * (E_eff/E0) ** (alfa)

for k in range(len(E_eff)):
    print(f'VGS = {VGS_vec[k]} V // E_eff = {E_eff[k]/1000:.3f} kV/cm // mu_eff = {mu_eff[k]:.3f} cm^2/Vs')

ID_mov = np.zeros((len(VGS_vec), len(VDS_vec)), dtype = float)

for k, VGS in enumerate(VGS_vec):
    Vds_sat = (VGS - VT)/m
    mu = mu_eff[k]

```

```

ID_mov[k, :] = mu * Cox * W/L * (VGS - VT - (m/2) * VDS_vec) * VDS_vec

idx_sat = VDS_vec > Vds_sat
ID_sat = mu * Cox * W/L * 1/(2 * m) * (VGS - VT)**2
ID_mov[k, idx_sat] = ID_sat

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, ID_mov[k, :] * 1000, fmt_array[k], label = r'\(V_{GS} = \) + f\{VGS\_vec[k]\} V')
    plt.plot(VDS_vec, ID_cuad[k, :] * 1000, 'k--')
plt.plot(VDS_vec, ID_cuad[0, :] * 1e3, 'k--', label = r'\(I_{D(original)})\')
plt.grid()
plt.xlabel(r'\(V_{DS}\) [V]\')
plt.xlim(0, VDD)
plt.ylabel(r'\(I_{D}\) [mA]\')
plt.legend()
plt.tight_layout()
plt.savefig("parteII/degradacion_movilidad.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

Ec = 27000 #campo crítico en V/cm
ID_velsat = np.zeros((len(VGS_vec), len(VDS_vec)), dtype = float)

Vds_saturacion = L * Ec * (np.sqrt(1 + 2 * (VGS_vec - VT)/(m * L * Ec)) - 1)
ID_saturacion = np.zeros_like(VGS_vec, dtype = float)

for k, VGS in enumerate(VGS_vec):
    ID_velsat[k, :] = (Cox * mu_eff[k] * W/L)/(1 + VDS_vec/(L * Ec)) * (VGS - VT - m/2 * VDS_vec) * VDS_vec

    Vds_sat = Vds_saturacion[k]
    ID_saturacion[k] = (Cox * mu_eff[k] * W/L)/(1 + Vds_sat/(L * Ec)) * (VGS - VT - m/2 * Vds_sat) * Vds_sat

    idx_sat = VDS_vec > Vds_sat
    ID_velsat[k, idx_sat] = ID_saturacion[k]

for k, VGS in enumerate(VGS_vec):
    plt.plot(VDS_vec, ID_velsat[k, :] * 1000, fmt_array[k], label = r'\(V_{GS} = \) + f\{VGS\} V')
    plt.plot(VDS_vec, ID_mov[k] * 1000, 'k--')

plt.plot(VDS_vec, ID_mov[0, :] * 1e3, 'k--', label = r'\(I_{D(movilidad)})\')
plt.plot(Vds_saturacion, ID_saturacion * 1000, 'rx', label = 'Saturación')
plt.grid()
plt.xlabel(r'\(V_{DS}\) [V]\')
plt.xlim(0, VDD)
plt.ylabel(r'\(I_{D}\) [mA]\')
plt.legend()
plt.tight_layout()
plt.savefig("parteII/velocidad_saturación.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

I_final = ID_dibl + factor * ID_velsat
#ID_velsat incluye degradación de la movilidad y saturación de la velocidad de arrastre
#el término factor incluye EMLC
#sumo ID_dible

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, I_final[k] * 1000, fmt_array[k], label = r'\(V_{GS} = \) + f\{VGS\_vec[k]\} V')

#plt.plot(VDS_vec, ID_cuad[3] * 1000, 'k--', label = r'\(I_{D}\) sin efectos')

```

```

plt.grid()
plt.xlabel(r' $\mathcal{E}V_{DS}$  [V]')
plt.xlim(0, VDD)
plt.ylabel(r' $\mathcal{E}I_D$  [mA]')
plt.legend()
plt.tight_layout()
plt.savefig("parteII/todos_juntos.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

dI_dDS = np.gradient(I_final, VDS_vec, axis=1)

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, dI_dDS[k] * 1e3, fmt_array[k], label = r' $\mathcal{E}V_{GS} = \mathcal{E}' + f'\{VGS\_vec[k]\} V'$ )

plt.grid()
plt.xlabel(r' $\mathcal{E}V_{DS}$  [V]')
plt.xlim(0, VDD)
plt.ylabel(r' $\mathcal{E}g_o = dI_D/dV_{DS}$  [mS]')
plt.legend()
plt.tight_layout()
plt.show()

for k in range(len(VGS_vec)):
    plt.plot(VDS_vec, 1/dI_dDS[k], fmt_array[k], label = r' $\mathcal{E}V_{GS} = \mathcal{E}' + f'\{VGS\_vec[k]\} V'$ )

plt.grid()
plt.xlabel(r' $\mathcal{E}V_{DS}$  [V]')
plt.xlim(0, 0.2)
plt.ylabel(r' $\mathcal{E}r_o = 1/g_o$  [ $\Omega$ ]')
plt.ylim(250, 2000)
plt.legend()
plt.tight_layout()
plt.savefig("parteII/ro.pdf", format = "pdf", bbox_inches = 'tight')
plt.show()

```